

Let Clickstream Talk: A Graph Neural Network Approach to Sales Forecasting^{1,2}

Rong Liu

Department of Information Systems and Business Analytics, College of Business, Florida International University, roliu@fiu.edu, Corresponding author.

Zihan Chen

School of Business, Stevens Institute of Technology, zchen61@stevens.edu.

Denghui Zhang

School of Business, Stevens Institute of Technology, dzhang42@stevens.edu.

Feng Mai

Department of Business Analytics, Tippie College of Business, University of Iowa, feng-mai@uiowa.edu.

Xuying Zhao

Department of Information and Operations Management, Mays Business School, Texas A&M University, xzhao@mays.tamu.edu.

Abstract: This paper proposes ForecastClickGraph, a deep learning framework that extracts cross-product relationships and demand information from clickstream data for probabilistic sales forecasting. ForecastClickGraph models stages in consumer shopping journeys as a dynamic graph, where nodes represent individual product-stage combinations (such as product A viewed or added to cart), and directed edges capture the transitions between stages. A customized graph neural network learns product representations that incorporate both product-level temporal patterns and cross-product associations. Another graph-based module further detects demand spikes using early signals from related products. ForecastClickGraph cross-learns demand patterns across large-scale product catalogs and estimates demand distributions through quantile regression. Extensive experiments on a real-world dataset show that ForecastClickGraph outperforms state-of-the-art benchmark models by 10-28% in forecast accuracy, with particularly strong performance in predicting promotional sales bursts. It also yields superior probabilistic forecasts with substantially lower quantile loss and improved calibration relative to benchmarks.

Key words: sales forecasting, graph neural network, clickstream, product relationship, sales burst

¹ This paper is accepted for publication in Production and Operations Management. DOI: [10.1177/10591478261490953](https://doi.org/10.1177/10591478261490953).

² This paper is the Accepted Manuscript version of the article. It can only be used for non-commercial research purposes.

Let Clickstream Talk: A Graph Neural Network Approach to Sales Forecasting

Abstract

This paper proposes ForecastClickGraph, a deep learning framework that extracts cross-product relationships and demand information from clickstream data for probabilistic sales forecasting. ForecastClickGraph models stages in consumer shopping journeys as a dynamic graph, where nodes represent individual product-stage combinations (such as product A viewed or added to cart), and directed edges capture the transitions between stages. A customized graph neural network learns product representations that incorporate both product-level temporal patterns and cross-product associations. Another graph-based module further detects demand spikes using early signals from related products. ForecastClickGraph cross-learns demand patterns across large-scale product catalogs and estimates demand distributions through quantile regression. Extensive experiments on a real-world dataset show that ForecastClickGraph outperforms state-of-the-art benchmark models by 10-28% in forecast accuracy, with particularly strong performance in predicting promotional sales bursts. It also yields superior probabilistic forecasts with substantially lower quantile loss and improved calibration relative to benchmarks.

Keywords: sales forecasting, graph neural network, clickstream, product relationship, sales burst

1. Introduction

The rapid growth of e-commerce has made accurate sales forecasting increasingly critical for effective inventory management and order fulfillment. According to data from the Federal Reserve Bank of St. Louis, e-commerce sales in the United States grew from \$80 billion in 2015 to \$289 billion in 2024. Moreover, e-commerce retail sales as a percentage of total sales rose from 4.2% in 2010 to 15.9% in 2024 (FRED, 2024). The COVID-19 pandemic has further accelerated the shift to online shopping, with many brick-and-mortar retailers expanding their e-commerce presence to adapt to changing consumer behavior (Torry, 2020). A key advantage of online platforms is the rich clickstream data they generate, providing granular information on consumer browsing behavior prior to purchase. The literature has drawn increased attention to the operational value of clickstream data (Huang and Van Mieghem, 2014, 2013; Martínez-de-Albéniz et al., 2020). Furthermore, Haruvy et al., (2014) provide evidence that clickstream data contains extensive information about the search process and the consideration set of a consumer, which can be leveraged to better understand and predict consumer behavior in the e-commerce context.

However, existing clickstream-based forecasting methods primarily focus on individual product-level patterns (Huang and Van Mieghem, 2014, 2013; Martínez-de-Albéniz et al., 2020). The rich cross-

product relationships that influence consumer purchase decisions are overlooked. In online shopping journeys, consumers often browse multiple products, comparing substitutes and seeking out complementary items before making a final purchase (Shocker et al., 2004). The order and duration of these product views provide valuable signals of consumer preferences and interests. For example, frequent co-views of two products may indicate substitutability, while views of one product frequently leading to purchases of another suggest complementarity. Uncovering and leveraging these latent product associations is key to improving forecast performance, particularly for retailers with large and diverse product catalogs. In other words, pooling information across related products should enhance forecast accuracy and efficiency compared to traditional single-product models.

The importance of capturing cross-product relationships is especially pronounced during promotional periods, such as Black Friday/Cyber Monday and other holidays, and during significant external events that alter consumer purchasing patterns. In either scenario, catalysts such as price discounts, product recommendations, or shifting consumer behaviors can drive sudden spikes in demand and induce cross-product spillover effects not present in historical data. For example, a steep discount on gaming consoles may stimulate demand for associated games and accessories. Failure to anticipate and incorporate these cross-product dynamics can lead to substantial forecast errors and sub-optimal inventory decisions. Demand forecasting during these volatile periods is particularly challenging due to the sparsity and irregularity of such events, which generate bursty sales patterns that deviate significantly from historical norms. Yet these intensive periods often account for a disproportionate share of annual sales and profits for online retailers (CNBC, 2023).

To address these challenges, we propose ForecastClickGraph, a novel graph-construction approach to learning from clickstream that captures both intra- and inter-product transitions during consumer shopping journeys to generate more accurate, granular demand forecasts. The core innovation of our approach is modeling aggregated consumer shopping journeys as a dynamic graph. In this graph, each node represents a specific product at a particular stage in the shopping journey, such as a product being viewed, added to cart, or purchased, with associated time-series attributes capturing both quantity and burstiness patterns. Directed edges between these product-stage nodes capture the transitions between stages, with edge weights representing transition frequencies that reveal nuanced relationships beyond simple co-occurrence. By applying graph neural networks (GNNs), a class of deep learning models designed for graph-structured data (Zhang et al., 2022), we learn product representations that incorporate both product-level temporal patterns and cross-product associations. To efficiently model the most relevant product relationships in large catalogs and capture temporal shifts in product interactions, we tailor graph attention to product-stage transitions leading to purchase and add a novel burst-prediction module that adaptively captures sudden spikes in demand based on early signals from related products.

Importantly, this graph structure evolves daily to reflect real-time changes in consumer behavior, particularly during promotions.

ForecastClickGraph offers distinct advantages over traditional time series and machine learning models commonly employed in operations management (OM) literature. While most models make predictions based on the feature vectors of individual products (Bastani et al., 2022), ForecastClickGraph leverages GNNs to efficiently learn from the interconnected patterns across multiple products, thereby capturing the complex dependencies and relationships that influence demand. This approach enables ForecastClickGraph to uncover implicit associations between products. In addition, ForecastClickGraph demonstrates superior scalability compared to multivariate time series models, which struggle to accommodate the high dimensionality and granularity of clickstream data (Zhang and Zhao, 2010). With customized GNN modules, ForecastClickGraph allows each product to attend only to the related neighbors that influence its sales and encodes their information into low-dimensional features. Thus, ForecastClickGraph maintains its efficacy and tractability even in the presence of extensive cross-sectional and temporal data across vast product catalogs.

Moreover, ForecastClickGraph addresses several key challenges in sales forecasting that are not adequately handled by existing machine learning-based methods. To tackle the issue of large-scale product catalogs and insufficient sales history, ForecastClickGraph enriches cross-learning (Salinas et al., 2020) with advance demand information and burstiness features learned across multiple products. This approach is coupled with quantile regression to provide probabilistic forecasts that capture the uncertainty in demand predictions – a crucial feature for effective inventory management. Also, ForecastClickGraph explicitly models the nonlinear, multi-stage nature of online shopping journeys by capturing the dynamic intra- and inter-product transitions that occur as customers browse and purchase items. Finally, ForecastClickGraph introduces a novel burst prediction module that explicitly forecasts the dramatic spikes in demand often observed during promotional events. By aggregating early demand signals from related products, this module anticipates and adjusts for upcoming bursts in sales, thus addressing a major shortcoming of traditional methods that struggle with high dimensionality of promotion-related variables.

Through extensive experiments on real-world data from a major e-commerce platform, we demonstrate the superior performance of ForecastClickGraph compared to a comprehensive set of state-of-the-art time series forecasting benchmarks, including DeepAR (Salinas et al., 2020), Informer (Zhou et al., 2021), NHITS (Challu et al., 2023), and dynamic graph neural network models like GC-LSTM (Chen et al., 2022) and T-GCN (Zhao et al., 2020). ForecastClickGraph achieves a 10-28% improvement in MSE. The performance gains are particularly pronounced during promotional periods, where ForecastClickGraph’s burst prediction module enables it to swiftly detect and respond to sudden spikes in demand. Ablation studies confirm the importance of the graph modeling and burst prediction components

in driving ForecastClickGraph’s superior performance. Furthermore, an evaluation of the probabilistic forecasts shows superior calibration, which provides a more reliable foundation for inventory decisions.

In summary, this paper makes the following contributions. First, we propose ForecastClickGraph, a novel graph-construction approach for cross-learning demand forecasts from clickstream data. The main contributions are a product-stage graph and a burst-prediction module, which together model relevant product associations and adapt to changing demand patterns. Second, our work enriches the literature on clickstream analysis by introducing novel uses of clickstream: capturing dynamic product relationships and detecting early signals for sales bursts. While our approach is inspired by established techniques such as GNN, it is specifically tailored to the challenges of probabilistic sales forecasting in dynamic e-commerce environments (Abbasi et al., 2024). Third, through comprehensive benchmarking on a large-scale e-commerce dataset, we demonstrate ForecastClickGraph’s superior performance, particularly in capturing bursty dynamics during promotions. Finally, our framework is particularly suited for marketplace and intermediary platforms, where access to sellers’ proprietary pricing and promotion information is limited. By leveraging real-time behavioral signals and observed sales dynamics from clickstream data, which implicitly reflect heterogeneous and often unobservable seller actions, our approach provides a robust and practical solution for demand forecasting under information constraints.

2. Background and Literature Review

Our study draws upon several domains in the literature, including sales forecasting on e-commerce platforms, the analysis of clickstream data and online shopping journeys, promotional effects, and advanced forecasting models. In this section, we introduce basic concepts in each domain and survey related work in the literature.

2.1. Sales Forecasting for E-commerce Platforms and Cross Learning

According to Tian et al. (2018), e-commerce platforms can be categorized into three primary types based on their business models. *Reseller* platforms, such as Bestbuy.com and Macys.com, purchase products from suppliers and resell them to consumers. *Intermediary marketplaces* (or simply *marketplaces*), like Tmall and eBay, allow suppliers to list and sell their products directly to consumers. *Hybrid* platforms, like Amazon.com and Walmart.com, act as resellers for some products while also serving as marketplaces for others. Reseller platforms have full control over sales decisions, including pricing and promotional strategies, and can aggregate data across products to improve sales forecasting. In contrast, on marketplaces and hybrid platforms, these decisions are managed by individual suppliers. Strategic information, such as pricing policies and promotion schedules, is often considered proprietary and not shared across suppliers, limiting collective forecasting capabilities. Our study specifically addresses sales forecasting under such information access constraints in these platform settings.

Sales forecasting has been studied extensively in the OM literature. In general, sales forecasting relies on two types of information: (1) historical sales data often represented as time series, and (2) contextual information relevant to consumers' purchase behaviors (Abolghasemi et al., 2020b). Previous studies have examined a variety of contextual information, including product complementarity and substitution relationships (Z.-Y. Chen et al., 2023), clickstream data (Huang and Van Mieghem, 2014), marketing campaigns (Fildes et al., 2022), intra- or inter-category sales promotions (Abolghasemi et al., 2020b), product reviews (G. Chen et al., 2023), online search traffic of competitive product pre-release (Schaer et al., 2022), and even weather (Steinker et al., 2017). While these contextual factors have been studied in isolation, accurate sales forecasting often requires integrating multiple factors (e.g., promotions and product relationships) and understanding their joint effects (e.g., the spillover effects of promotions on related products) (Qi et al., 2019).

In practice, individual sales forecasting models, such as regression and Prophet (Taylor and Letham, 2018), are created for each stock-keeping unit (SKU) based on its historical sales data and other features. This SKU-level sales forecast is essential for daily stock replenishment and distribution operations (Fildes et al., 2022). When an SKU has accumulated an abundant sales history with regularity and a stable trend, such models can perform well in predicting future demand. However, their accuracy often deteriorates when applied to newly launched products with limited sales history or to seasonal products with short lifecycles (Cohen et al., 2022). Moreover, promotional events can significantly influence consumer behaviors and market demands (Abolghasemi et al., 2020b; Fildes et al., 2022), leading to non-stationary time series or high volatility. In addition, the large number of SKUs makes training an individual forecasting model for each SKU computationally expensive (Spiliotis et al., 2022).

Alternatively, *cross-learning* can enhance forecasting performance without significantly increasing the computational cost (Salinas et al., 2020; Spiliotis et al., 2022). This method pools sales data across products to train a single forecast model for multiple SKUs. With enhanced data availability, the model can capture common patterns across time series, adapt to structural changes (e.g., due to promotions), and thereby improve forecasting accuracy (Fildes et al., 2022). For example, Salinas et al. (2020) demonstrated the potential of cross-learning for thousands of SKU time series.

In practice, cross-learning models often estimate the *probabilistic demand distributions* for individual products in two approaches. Unlike a point forecast, which offers a single demand expectation, probabilistic forecasts capture a range of possible demands and allow decision makers to evaluate the risks associated with downstream tasks, such as replenishment and inventory management (Petroopoulos et al., 2022). The first approach assumes that a parametric probability distribution (e.g., Gaussian) generates forecasts and directly estimates the distribution parameters (e.g., mean and variance) to minimize the error between actual sales and samples drawn from the distribution. For instance, DeepAR (Salinas et al.,

2020) assumes Gaussian for real-valued data and negative-binomial for count data. However, assuming a fixed distribution across thousands of time series can be problematic, as product demands may follow diverse distributions. An alternative is quantile regression (Koenker and Bassett Jr, 1978), which does not rely on distributional assumptions. This approach directly produces sales forecasts at selected quantiles. Studies have shown that integrating quantile regression with deep learning models is particularly effective for demand forecasting (Park et al., 2022; Petropoulos et al., 2022; Wen et al., 2017).

Moreover, it is desirable to generate *multi-step forecasts*, i.e., demand estimates over the interval $[T, T + H]$, given a product’s sales history up to time T . Such forecasts can facilitate more informed operational planning, such as estimating demand fluctuations over an entire promotion period. The literature outlines two strategies for generating multi-step forecasts. The recursive strategy often employs autoregressive models to generate one forecast at a time and then feed it back into the model to generate the next. A drawback of this strategy is that forecast errors can be propagated and accumulated over the forecast horizon (Petropoulos et al., 2022). To mitigate this, a direct strategy, which produces multi-step forecasts at one time, is preferred (Petropoulos et al., 2022).

Overall, the key challenge in cross-learning is effectively extracting rich features that capture diverse patterns in massive product sales time series (Semenoglou et al., 2021). Meanwhile, these features should remain sufficiently expressive to detect shifts in individual product demand patterns over a multi-step forecasting horizon.

2.2. Clickstream Data and Online Shopping Journeys

Clickstream data not only provides advance demand information (Huang and Van Mieghem, 2013) but also allows firms to learn about consumer behaviors and preferences (Ding et al., 2015; Ferreira et al., 2022; Martínez-de-Albéniz et al., 2020). In general, an online shopping journey consists of three stages: *viewing* products,¹ creating shopping carts (referred to as *cart* for simplicity), and making *purchases* (Sahni et al., 2019). However, the journey is not necessarily linear, as consumers can be influenced by exogenous factors such as promotions and advertisements. Customers usually start at the viewing stage, and they may jump to other stages, move back and forth between stages, or terminate their sessions at any point. Figure 1(a) shows two sessions of a user’s online shopping journey. In Session S1, the customer showed interest in a smartphone and a pair of headphones both from the Apple brand but ultimately chose a smartphone from a competing brand. Afterward, the user appeared to remain interested

¹ View events can be triggered by user scrolling behaviors or interactions with a product. For example, a view event can be recorded when a product listed on a page enters a user’s viewpoint during scrolling (Zhang and Wang, 2024). When the page displays multiple products simultaneously, multiple view events can be triggered within a short time window. User interactions like clicking on a product link or button also triggers a view event. For example, if a product features a one-click purchase option, clicking this button would generate a view event immediately followed by a purchase event.

in buying headphones, likely as an accessory to the new smartphone. In Session S2, the customer initially browsed JBL headphones but eventually purchased Apple headphones. The stage transitions in the journeys provide opportunities to discover customers’ hidden preferences for products. Figure 1(b) shows a graphical representation of these two sessions, where each *SKU-stage* is shown as a node and the transition between two stages (e.g., co-clicks) as an edge. It is worth noting that these sessions involve not only *intra-product transitions*, but also *inter-product transitions* surrounding SKU C. These transitions may indicate complementary or substitute products of SKU C. Hence, product relationships naturally come into play in the clickstream graph.

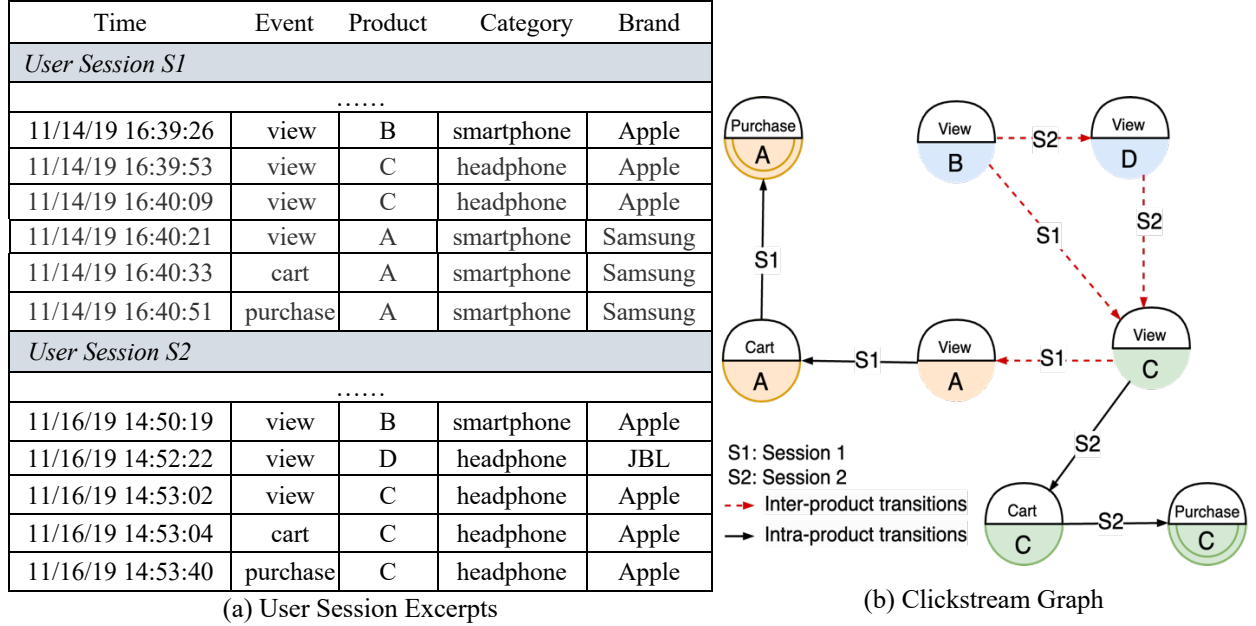


Figure 1: Examples of Online Shopping Journeys and Clickstream Graphs

A closely related stream is recommender systems, which also leverages clickstream or other sequential behavioral data but with a different objective: recommending or ranking products/content to influence consumer choices and improve engagement or conversion. In OM, recommender systems are studied both as demand-shaping technologies and as decision/optimization problems. For example, Fleder and Hosanagar (2009) analyze how recommender design can shift aggregate sales concentration and diversity; Demirezen and Kumar (2016) and Ettl et al. (2020) integrate inventory considerations into recommendation decisions; Besbes et al. (2016) formulate content recommendation as a dynamic optimization problem along users’ browsing paths. Recent field experiments quantify how recommendation design choices affect demand and consumer behavior (e.g., Adamopoulos and Todri, 2025; Long et al., 2025; Wan et al., 2025). At the same time, other studies use clickstream data to actively intervene in the shopping experience or to quantify how platform design affects purchases, for instance, by adapting web pages based on real-time intent (Ding et al., 2015), ranking assortments to maximize

engagement (Ferreira et al., 2022), measuring spillover effects of online reviews (Kwark et al., 2021), or evaluating one-click buying (Unal and Park, 2023).

From a modeling perspective, recommender systems traditionally build on collaborative filtering and matrix factorization (Adomavicius and Tuzhilin, 2005; Koren et al., 2009), and modern systems combine these ideas with deep sequential and graph representation learning. Transformer-based sequential recommenders (e.g., BERT4Rec (Sun et al., 2019)) encode user/session histories, while graph-based recommenders learn embeddings from user-item bipartite graphs or session graphs to predict user-item interactions (see Wang et al. (2021) and Wu et al. (2023) for surveys). These models typically aim to predict a user’s next click/purchase or produce a ranked list of items. In contrast, our focus is multi-step, probabilistic sales forecasting: predicting aggregate SKU-level demand distributions over a future horizon for operational planning (e.g., inventory and service level decisions). We use clickstream data to infer dynamic cross-product relationships and early demand signals (views/carts) that precede purchases, which is particularly important during promotion-driven demand shocks. Appendix 1 provides a structured survey of graph models using clickstream data (Section A1-1) and a detailed comparison between recommendation and our forecasting task (Section A1-2).

The literature has noted the importance of clickstream data for demand modeling. As illustrated in Figure 1(a), the time lag between a consumer’s first product view (e.g., Apple headphones) and the eventual purchase event may span several days, providing retailers with advance signals to respond to potential demand shifts. Importantly, such information is obtained “without asking” customers (Huang and Van Mieghem, 2014). Martínez-de-Albéniz et al. (2020) develop a hierarchical model to predict sequential shopping decisions using clickstream data and show its effectiveness in forecasting volatile demand during flash sales. Huang and Van Mieghem (2013) demonstrate that incorporating clickstream data improves forecast accuracy and reduces inventory holding and backorder costs by about 5%. They further demonstrated that clickstream data on non-transactional websites can benefit the prediction of the propensity, amount, and timing of offline purchase orders (Huang and Van Mieghem, 2014).

To summarize, research in recommendation systems focuses on individual consumer behaviors to optimize conversion within a single user session. In contrast, sales forecast targets aggregated demand across consumers over a multi-step forecast horizon that may span multiple user sessions. Moreover, clickstream studies in OM usually focus on intra-product transitions while overlooking inter-product transitions. However, inter-product transitions can provide valuable signals for demand estimation during promotions, as promotional activities often generate substantial spillover effects across related products.

2.3. Holiday Seasons, Promotions and Bursting Sales

The holiday season is characterized by a plethora of promotional events launched either jointly by platforms and sellers or independently by each party (Cheng et al., 2023). Joint promotions, such as Black

Friday, are often initiated by platforms, and sellers can decide whether to participate. Meanwhile, sellers on marketplaces or hybrid platforms can independently implement their own promotions, either within a brand or category (e.g., buy one get one free), or across multiple categories (e.g., clearance sale) (Cheng et al., 2023). These seller-led strategies are often considered as proprietary and may not be disclosed to the platforms or other sellers in advance (Hagiu and Wright, 2015).

Promotions can generate complex spillover effects. It is well known that promoting a product may increase the sales of its complements but negatively affect its substitutes due to brand switching (Fildes et al., 2022). These effects are particularly prominent on e-commerce platforms where users can easily compare substitute products at little cost (Z.-Y. Chen et al., 2023). Research also indicates that promotions can significantly change consumer behaviors, with effects extending even to sellers who do not offer promotions themselves (Hagiu and Wright, 2015; Tian et al., 2018; Zhang et al., 2020). For example, Zhang et al. (2020) find that shopping cart promotions increase the proportion of products that customers add to their carts, but decrease the prices they subsequently pay, regardless of whether the products are on promotion. Moreover, spillover effects may also be shaped by factors beyond promotions, including intrinsic product characteristics, brand identity, and latent attributes such as perceived quality, price elasticity, response to promotions, and consumption context (Voleti et al., 2015). As a result, identifying which products are affected by spillover effects is particularly challenging on large platforms like Amazon, which offers over 300 million products with rapid turnover (Tian et al., 2018).

Promotions and other exogenous shocks, such as viral product fads, can dramatically change the demands not only for target products but also for related items due to the spillover effects, posing substantial challenges in sales forecasting and operations management. First, promotions often cause sudden spikes in sales, but their timing and scale are difficult to forecast. For instance, a grand promotion can increase sales up to ten times (Chi et al., 2024), intensifying the pressure on warehousing and distribution systems. This issue is particularly acute in marketplaces or hybrid platform settings, where individual sellers can independently schedule promotions (Cheng et al., 2023), making it nearly impossible for other sellers to anticipate the timing and impact of potential spillover effects before the promotions start. Second, irregular promotion strategies can cause significant fluctuations in demand. Since promotion events are sparse and often non-repetitive, it becomes even more difficult to learn the underlying demand patterns (Feng and Shanthikumar, 2018). This challenge is exacerbated when products lack sufficient sales history, such as new products (Baardman et al., 2018) or seasonal items (Wolters and Huchzermeier, 2021). Third, due to consumer stockpiling in response to the price reduction, often the demands for promoted products are reduced post promotions (Hewage et al., 2022). Finally, studies have indicated that promotions can lead to “herd behavior” where a consumer’s purchasing decisions are

induced by the high purchasing volume of other people, rather than the discounts (Chi et al., 2024). Hence, promotions can compromise the accuracy of sales forecasts.

Existing forecast methods attempt to incorporate promotion-related variables, such as promotion schedules and discount schemes for target and other related products (Abolghasemi et al., 2020b; Huang et al., 2014; Ma et al., 2016). However, these variables can be difficult to obtain, as seller-led promotions are often considered proprietary information (Hagiu and Wright, 2015). Even when such data is available, incorporating cross-product promotional effects substantially increases dimensionality, raising the risk of model overfitting due to limited data. To mitigate this issue, prior studies employ continuous shrinkage methods to identify the most influential predictors (Huang et al., 2014; Ma et al., 2016). Alternatively, due to the model complexity and the cost of collecting promotion-related variables, many retailers use a simple statistical model supplemented by human judgmental adjustments (Fildes et al., 2019). However, the adjustments are often subject to human biases (Fildes et al., 2022). Moreover, most methods ignore the impact of promotions on demand during the post-promotional period (Hewage et al., 2022).

In summary, existing promotion studies suffer from difficulties in identifying product relationships and spillover effects at scale, often rely on proprietary promotion information, and are susceptible to bias from manual interventions. These challenges necessitate a dynamic, automated approach capable of capturing the effects of promotions on related products and inferring the timing of sales bursts based on real-time clickstream data.

2.4. Forecast Models and Deep Learning

Traditional techniques for demand forecasting range from univariate time series analysis, such as ARIMA and Prophet (Taylor and Letham, 2018), to multivariate time series regression like vector autoregression (Abolghasemi et al., 2020a; Spiliotis et al., 2022; Wolters and Huchzermeier, 2021). Researchers have found that simple time series techniques generally can perform well during periods without promotions, while advanced models are needed to incorporate promotional information for better accuracy during promotions (Fildes et al., 2022). Next, we review relevant forecast models.

Traditional machine learning and deep learning. Machine learning has been employed in sales forecasting with historical sales and contextual factors as input. Techniques include Support Vector Regression (SVR), regression trees, and multilayer perceptron networks (MLP) (Fildes et al., 2022; Spiliotis et al., 2022). These methods can estimate heterogeneous nonlinear relationships among input features. Stacked MLPs have been applied to extract complex characteristics of time series. For instance, NHITS (Challu et al., 2023) takes a residual architecture with multiple stacks of MLPs. Each stack learns a signal (e.g., trend) and passes the unexplained residuals to the next stack. Overall, these methods treat each product's time series as an observation and each time series data point as an independent feature. Thus, they lack the capability to identify deep temporal patterns or to capture cross-product relationships.

Sequence learning. Sequence learning models are designed to capture temporal dependencies within a sequence using memory or attention mechanisms. These models map one sequence to another, for example, transforming historical time series into multi-step sales forecasts, through an encoder-decoder architecture. Recurrent neural networks (RNNs) maintain hidden states as memory that are recursively updated and referenced throughout the sequence. For instance, DeepAR employs an auto-regressive RNN that takes lagged target values and covariates as inputs (Salinas et al., 2020). Alternatively, Transformer models (Devlin et al., 2019) and variants like Informer (Zhou et al., 2021), use attention mechanisms to selectively attend to relative parts of the sequence during processing.

Graph neural networks. Graph neural networks (GNNs), which model relational dependencies through graph structures, have been widely adopted to capture shopping journeys, product relationships, and user-product interactions (Wang et al., 2021). For example, GNNs have been used to infer product relationships from clickstream data (Hao et al., 2020; Yan et al., 2022; Yilmaz and Öğüdücü, 2021). These models treat products as nodes and specific product interactions, such as co-views and co-purchases, as edges. Each node selectively aggregates neighbors’ information through attention (e.g., GAT (Veličković et al., 2018) and Graph Transformer (Shi et al., 2021)) or through MLPs (e.g., GraphSage (Hamilton et al., 2017)) to generate product-relation embeddings. In recommendation systems, GNNs are used to model user-product interactions (e.g., collaborative filtering on user–item bipartite graphs) and session graphs (e.g., next-item prediction within a session) (Wang et al., 2021; Wu et al., 2023). A survey of GNNs for clickstream can be found in Appendix 1. Overall, existing studies focus on a narrow set of product relationships, such as co-view and co-purchase, while more nuanced relationships, e.g., purchase-after-view (e.g., one-click buy; Unal and Park, 2023) and purchase-after-purchase (e.g., repeated buy), remain unexplored. Moreover, these relationships are typically learned in offline settings and may fail to capture real-time product dynamics driven by ongoing promotions.

Dynamic graph neural networks. While sequence learning models capture temporal patterns within individual product time series and GNNs model relationships across products, combining the two is a natural direction for sales forecasting, as it allows for the integration of both temporal dynamics and inter-product relationships. *Dynamic* GNNs model such relationships by integrating RNNs with GNNs in various ways. For example, T-GCN (Zhao et al., 2020) first applies graph convolution to extract neighborhood features at each snapshot, which are then fed into an RNN as sequential inputs. In contrast, GConvGRU and GConvLSTM (Seo et al., 2018) take the hidden states of an RNN as node features for graph convolution. Similarly, GC-LSTM (Chen et al., 2022) incorporates RNN hidden states into graph convolution, while also utilizing the graph adjacency matrix within the RNN to update hidden states. Dynamic GNNs, particularly, a subclass named spatio-temporal graph models, have been widely applied in domains such as traffic forecasting and human motion modeling, where the underlying graph

structures, such as road networks, are typically predefined and relatively stable (Feng et al., 2025; Jin et al., 2023). In contrast, product graphs on e-commerce platforms are highly dynamic and complex due to frequent product entry and exit as well as nuanced interactions like co-cart or sequential promotions (e.g., buy-A-then-B). Moreover, most dynamic GNNs often assume stable temporal patterns and lack mechanisms to infer abrupt demand changes, especially when promotion signals are unknown or unavailable. More discussion on dynamic GNNs can be found in Appendix 2.

To summarize, our literature review identifies four key challenges or design requirements (R1-R4) in sales forecasting for e-commerce platforms, as shown in Table 1. (R1) Due to the massive products and high demand variability, probabilistic cross-learning models are preferred but require rich features to capture diverse demand patterns. (R2) Existing methods lack effective modeling of intra-/inter-product transitions for aggregated sales forecasting. (R3) Forecast models need to automatically discover dynamic, nuanced product relationships and assess complex promotional effects at scale. (R4) Since promotions may not be disclosed in advance, forecast models need to detect burstiness signals and infer the timing and magnitude of sales bursts in real time, without requiring prior knowledge of promotions.

Table 1: Summary of Design Requirements, Related Work, and Design Components

A. Design Requirements	B. Existing Methods and Limitations	C. Design Components
(R1) Large scale of products, limited sales history, and high demand variability	- Individual SKU-level forecast models are computationally expensive and inaccurate [1] - Cross learning pools thousands time series across products [2]; required to capture diverse patterns	Enriching cross learning with expressive features learned from related products to capture advance demand and burstiness features
(R2) Nonlinear online shopping journeys with dynamic intra-/inter-product transitions	- Recommendation systems focus on individual consumer behaviors within a user session [3], while sales forecasting targets aggregated demand across consumers and sessions - Aggregated clicks used as covariates without fully considering intra-/inter-product transitions [4, 5]	Unique graph construction that explicitly models shopping journey stages and intra-/inter- stage transitions; with aggregated time series data embedded at each node to capture temporal dynamics
(R3) Complex product relationships and promotional effects on related products	- Product relationships are manually captured [6]; difficult to scale on e-commerce platforms - A narrow set of inter-product relationships (e.g., co-view, co-purchase) are extracted from clickstream and represented as static graphs [7,8]	Automatically capturing a broad range of nuanced product relationships (e.g., co-cart, one click purchases) from intra-/inter-product transitions; learning advance demand from related products
(R4) Changed sales patterns and dramatic sales bursts due to promotions	- Assuming promotions are disclosed in advance, modeling them as features but suffering from high dimensionality [9,10,11] - Human adjustment of promotional effects but suffering from bias [6, 12]	Using a GNN to aggregate burstiness signals from related products and inferring the timing and scale of sales bursts in real time, without requiring prior knowledge of promotions
[1] Spiliotis et al., 2022; [2] Semoglou et al., 2021; [3] Wu et al., 2023; [4] Huang and Van Mieghem, 2013; [5] Huang and Van Mieghem, 2014; [6] Fildes et al., 2022; [7] Hao et al., 2020; [8] Yilmaz and ÖGüdücü, 2021; [9] Abolghasemi et al., 2020b; [10] Huang et al., 2014; [11] Ma et al., 2016; [12] Fildes et al., 2019		

3. Design Requirements and Forecast Architecture

To address these challenges, we propose a customized deep learning model named *ForecastClickGraph*, as shown in Figure 2(A). As summarized in Column C of Table 1, our model contains design components corresponding to the design requirements (R1-R4). To respond to R1, we propose a comprehensive cross-learning model with quantile regression to produce probabilistic forecasts for SKUs in a multi-step future horizon. The forecasts sum up three effects learned simultaneously. First, similar to existing methods (Salinas et al., 2020), our model learns time-dependent features from an SKU’s historical sales through an LSTM layer to generate a base forecast. Second, to address R2 and R3, we adapt a graph neural network to capture the dynamic interactions between products. Specifically, we model aggregated customer shopping journeys as graphs, where each node represents a product-stage combination (e.g., SKU A view), and edges denote intra-/inter-product transitions. With the graphs, we encode the neighbors leading to the purchase of a product into a feature for sales forecasts. Moreover, considering the high correlation between a node (e.g., SKU A purchase) and its neighbors (e.g., SKU A view), we use a custom-designed gate mechanism to regulate this feature and avoid inflating sales forecasts. Lastly, we address the design requirement R4 with a dedicated burst prediction component, which adjusts the sales forecasts based on the aggregated advance demand information from neighbors. Next, we formally define our forecast problem and introduce our notations, followed by a detailed description of each component.

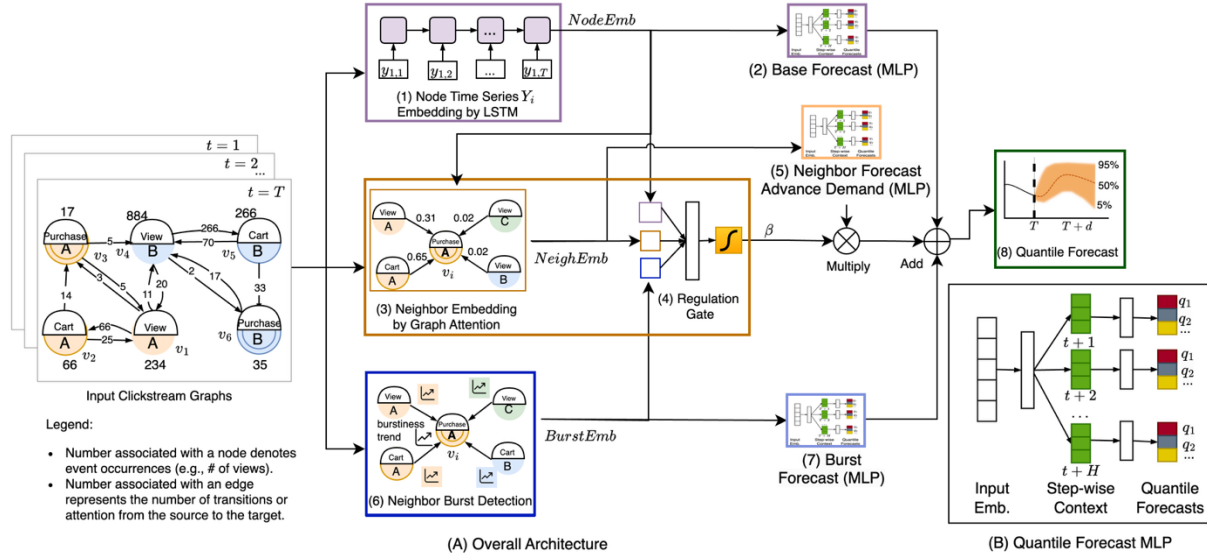


Figure 2: ForecastClickGraph: A Cross-learning Framework for Multi-step Probabilistic Forecasts

3.1. Problem Definition and Clickstream Data

Since we target forecasting daily SKU sales, we first aggregate user sessions by SKU-stage and by day and then model the aggregated data as a graph, as illustrated in Figure 3(A). We call each SKU-

stage an event and represent it as a node, with a node attribute denoting the number of the event occurrences. For instance, in graph g_1 , SKU A has been viewed (i.e., node v_1) for 146 times and added into carts (i.e., node v_2) for 40 times, and its sales (i.e., node v_3) volume is 9. Accordingly, a transition between SKU-stages is modeled as an edge, with a weight indicating the number of occurrences of this transition. For instance, in graph g_1 , for SKU A, out of 146 views, there are 40 intra-product transitions from “SKU A view” to “SKU A cart.” Additionally, 30 inter-product transitions from “SKU A View” to “SKU B View,” as some users proceed to view SKU B.

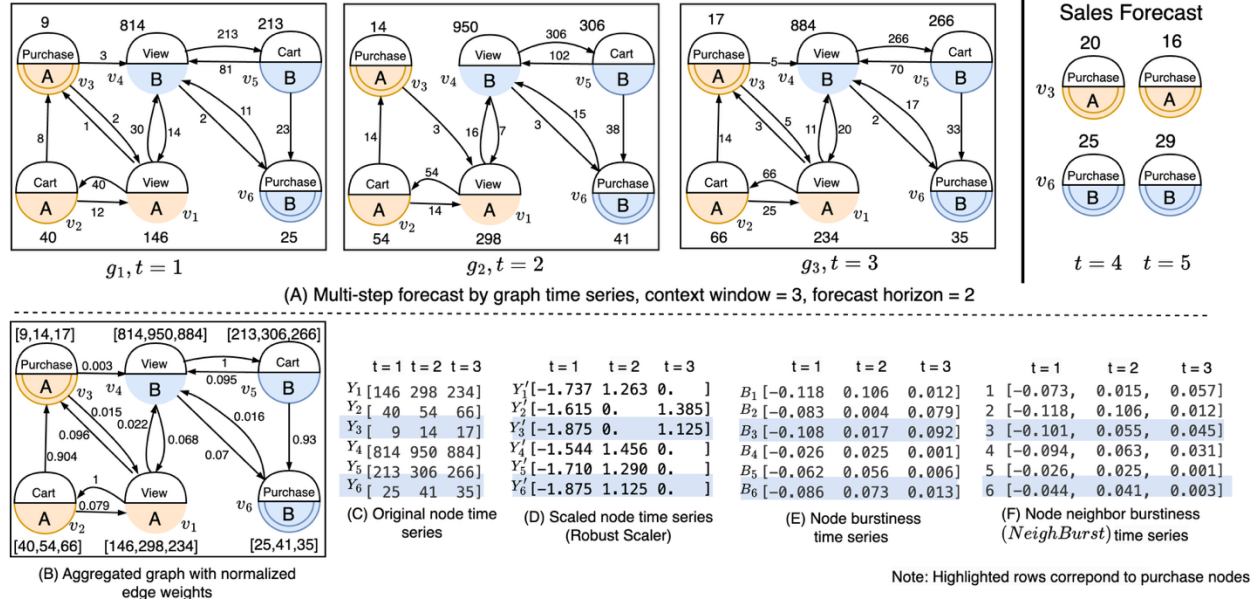


Figure 3: An Illustrative ForecastClickGraph Example

Our model takes a time series of graphs as input to forecast sales. Appendix 3 lists the notations. Formally, let $\mathbb{V} = (v_1, v_2, \dots, v_M)$ be the set of all nodes. The clickstream data over a context window $[1, 2, \dots, T]$ is represented as a sequence of timestamped graphs, $\mathbb{G} = (G_1, G_2, \dots, G_T)$, where each graph $G_t = (V_t, E_t)$ corresponds to time t . The node set is $V_t = \{(v_i, y_{i,t})\}_{i=1}^M$, where $y_{i,t}$ denotes the attribute of node v_i . The edge set is $E_t = \{(v_i, v_j, w_{i,j,t}) \mid w_{i,j,t} > 0\}_{i,j=1}^M$, with $w_{i,j,t}$ as the weight of the edge from nodes v_i to v_j . Each node v_i has a *node time series* $Y_i = \{(y_{i,t})\}_{t=1}^T$, capturing event occurrences over the context window. Accordingly, $W_{i,j} = \{(w_{i,j,t})\}_{t=1}^T$ represents the *weight time series* of the edge from v_i to v_j within the context window. For instance, in Figure 3(A), the time series of SKU A purchase (node v_3) is $Y_3 = (9, 14, 17)$. Figure 3(C) lists all node time series. Note that these time series may differ in scale. We apply the Robust Median Scaler (Rousseeuw and Croux, 1993) to normalize each time series, say $Y = \{y_1, y_2, \dots, y_n\}$, into $Y' = \{z_1, z_2, \dots, z_n\}$, where $z_i = \frac{y_i - \text{Median}(Y)}{\text{MAD}(Y)}$, $\text{MAD}(Y) = \frac{1}{n} \sum_{j=1}^n |y_j - \text{Median}(Y)|$. For

example, for Y_3 , the median is 14, and the MAD is $8/3$, and $Y'_3 = (-1.875, 0, 1.125)$. Figure 3(D) shows the scaled time series on a comparable scale across nodes.

To stabilize node transitions, we compute an aggregated graph by averaging edge weights across the context window, as shown in Figure 3(B). To account for scale differences, we normalize each edge weight by dividing it by the average quantity of the target node. We use this graph to find the neighbors of each node. For node v_i , we denote its upstream neighbors as $N_i^{Up} = \{v_j \mid w_{j,i,t} > 0\}$, and its downstream neighbors as $N_i^{Down} = \{v_j \mid w_{i,j,t} > 0\}$. For instance, as shown in Figure 3(B), the upstream neighbors of node v_3 are $N_3^{Up} = (v_1, v_2)$ and downstream neighbors $N_3^{Down} = (v_1, v_4)$.

Finally, we formally *define our forecast problem as a multi-step quantile forecast* (Wen et al., 2017). Given $\mathbb{G} = (G_1, G_2, \dots, G_T)$ and a node v_i in the graphs, our goal is to provide a probabilistic estimate of v_i 's time series in the next H steps (Equation 1), i.e.,

$$\text{Multi-step probabilistic forecast: } \mathcal{P}(y_{i,T+1}, y_{i,T+2}, \dots, y_{i,T+H} \mid \mathbb{G}) \quad (1)$$

Given a set of predefined quantiles Q , (e.g., 10%, 50%, 90%), for any $q \in Q$ and $1 \leq h \leq H$, we predict the conditional quantile $y_{i,T+h}^{(q)}$ of the target distribution, i.e.,

$$q\text{-Quantile forecast: } \mathcal{P}(y_{i,T+h} \leq y_{i,T+h}^{(q)} \mid \mathbb{G}) = q \quad (2)$$

For example, as illustrated in Figure 3(A), the target is to compute the probabilistic forecasts for the purchase of SKU A (v_3) and SKU B (v_6) over the forecast horizon $[4, 5]$.² Next, we describe the design components in our ForecastClickGraph framework shown in Figure 2.

3.2. Base Forecast by Recurrent Neural Network

Our model first generates a base forecast from the node time series. When a product has a stable sales history and no promotions are scheduled in the forecast horizon, the sales history alone can provide sufficient information for an accurate forecast (Fildes et al., 2019). Zhu et al. (2021) found that recurrent neural networks are effective in extracting temporal patterns and nonlinear features. Hence, we use an LSTM layer to encode each node time series into a *node embedding*. Given time series input Y_i , LSTM maintains internal states as “memory” and recursively updates the states as it processes each element, $y_{i,t}$. It parses $y_{i,t}$ using the states to extract new information, selectively updates the states, and then outputs a new vector representation for $y_{i,t}$. This new representation has incorporated information from $y_{i,t}$'s predecessors. In particular, the representation for the last element has encoded the information from the entire sequence of Y_i and thus can be used as an embedding of Y_i . For simplicity, we treat an LSTM

² While our model can forecast the time series corresponding to any product stage (e.g., views), to focus on sales forecast, we train the model to minimize the forecast errors corresponding to purchase nodes only.

network as a function. We apply this function to time series Y_i to generate an embedding for Y_i , denoted as $NodeEmb_i$ (Equation 3).

Next, the model produces multi-step forecasts based on $NodeEmb_i$. As discussed in Section 2.4., a direct strategy, which produces multi-step forecasts at one time, is preferred. We use an MLP network as suggested by Wen et al. (2017) to generate multi-step quantile forecasts directly (Equation 4). This MLP network is shown in Figure 2(B). For each step in the forecast horizon, we first project $NodeEmb_i$ to generate a step-specific context vector using a dense layer with parameter matrix $W_{context}$ and ReLU activation function. This context vector is then used as an input to another dense layer with trainable parameter W_{output} to output the quantile forecasts, $BaseForecast_i \in \mathbb{R}^{H \times |Q|}$ for the forecast horizon H and quantiles Q ($|Q|$ denotes the number of quantiles).

$$NodeEmb_i = LSTM(Y_i) \quad (3)$$

$BaseForecast_i = MLP(NodeEmb_i)$, where the function MLP can be decomposed as:

$$\begin{aligned} Z_{context,i} &= ReLU(W_{context} NodeEmb_i), \\ BaseForecast_i &= W_{output} Z_{context,i} \end{aligned} \quad (4)$$

3.3. Burst Prediction by Graph Neural Network

While the RNN component captures a product's normal sales dynamics to generates baseline forecasts, our framework also includes a dedicated burst detection module to address promotion-driven deviations from this norm, which are difficult to predict using only a product's own sales history. In contrast, clickstream data provides valuable signals from related products that help anticipate future sales bursts. To illustrate, Figure 4 shows the rise of view and cart events of two related products starting on November 13. This rise leads to the inception of purchase event bursts two days later. Thus, if bursts in these neighbor nodes can be identified, they can serve as effective early signals for predicting future sales bursts. Moreover, signals aggregated from neighbors can be more reliable and informative. A burst in a single time series may reflect noise, whereas simultaneous bursts from more neighbors likely indicate a platform-wide promotion. Thus, we employ a graph neural network to aggregate burstiness indicators from the time series of neighbor nodes and use these early signals to adjust baseline forecasts, enabling timely correction during promotion periods.

We first calculate the burstiness score of each interval in a node time series Y_i to form a new time series (B_i), capturing the scale and timing of burstiness (Equation 5). Motivated by Lappas et al. (2009), we extract burstiness scores from input time series as features. Burstiness is referred to as the deviation of a situation from the expected behavior baseline. Given a time series $(x_1, x_2, \dots, x_T)$, Lappas et al. (2009) measures burstiness of a segment $[r, l] \subseteq [1, T]$ as $\frac{\sum_{t=r}^l x_t}{\sum_{t=1}^T x_t} - \frac{l-r+1}{T}$, where $\frac{l-r+1}{T}$ expresses the expected

behavior baseline, while $\frac{\sum_{t=r}^l X_t}{\sum_{t=1}^T X_t}$ represents the observed fraction. For example, in Figure 3(C), for the node time series $Y_3 = (9, 14, 17)$, the burstiness scores of each interval is $B_3 = (-0.108, 0.017, 0.092)$. The last two elements receive higher scores because their observed fractions exceed the expected baseline of $1/3$. Figure 3(E) shows the burstiness time series of all nodes.

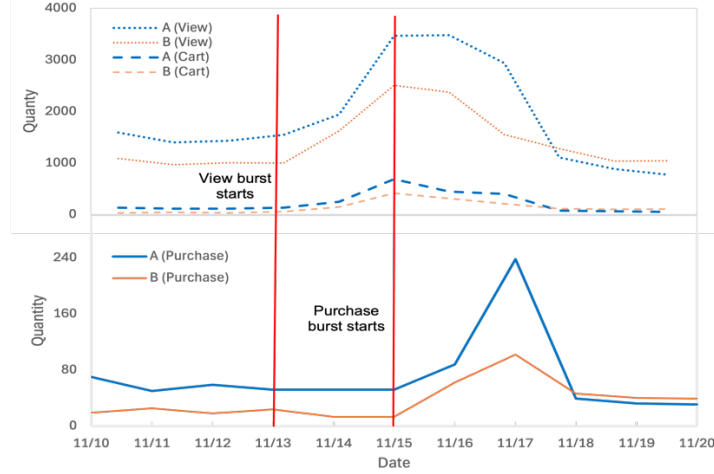


Figure 4: Advance Demand Information from View and Cart Events

Next, for each node, we average the burstiness time series across its upstream neighbors to compute a neighbor burstiness context for each node i ($NeighBurst_i$) (Equation 6). The intuition is that the burstiness in upstream nodes (e.g., view and cart) can lead to increased activities in downstream nodes (e.g., purchase). The resulting $NeighBurst_i$ itself is also a time series, capturing the scale and timing of burstiness in the node’s neighborhood. To illustrate, consider Figure 3(B), where the purchase node v_3 has two upstream nodes, v_1 (view) and v_2 (cart). The neighbor burstiness for v_3 is the average of the burstiness time series of these two nodes, i.e., $NeighBurst_3 = \left(\frac{B_1+B_2}{2}\right) = (-0.101, 0.055, 0.045)$. This series shows a burst beginning at $t = 2$, suggesting a spike in purchase may follow the burst. Figure 3(F) shows the neighbor burstiness time series of all nodes. Note that neighbor burstiness is always computed in the source-to-target direction, aggregating information from upstream nodes. When a node acts as a source, it contributes to the neighbor burstiness of its downstream nodes. For example, in Figure 3(B), v_3 is also a source node of v_1 , reflecting a view-after-purchase pattern. The neighbor burstiness of v_1 is the average burstiness time series of v_2, v_3 , and v_4 , i.e., $NeighBurst_1 = \left(\frac{B_2+B_3+B_4}{3}\right) = (-0.073, 0.015, 0.057)$.

Then, $NeighBurst_i$ is transformed through a dense layer to generate a burstiness embedding $BurstEmb_i$ (Equation 7). Finally, with $BurstEmb_i$ as an input, we generate burst quantile forecasts, $BurstForecast_i \in \mathbb{R}^{H \times |Q|}$ for node i using a new instantiation of the MLP network shown in Figure 2(B) (Equation 8). Since the input burstiness scores are scale-free, the forecasts only indicate projected deviation scales (in percentage) from the current trend. We use the hyperbolic tangent activation ($\tanh$) to

constrain the forecasts to the range $[-1, 1]$. The current trend can be estimated using exponential moving average (denoted as Emv_i) with a smoothing factor (λ), for example, 0.7.

$$\text{Given } Y_i = (y_{i,1}, y_{i,2}, \dots, y_{i,T}), B_i = \{b_{i,t}\}_{t=1}^T, \text{ where } b_{i,t} = \frac{y_{i,t}}{\sum_{k=1}^T y_{i,k}} - \frac{1}{T} \quad (5)$$

$$NeighBurst_i = \left\{ \frac{1}{|N_i^{UP}|} \sum_{k \in N_i^{UP}} b_{k,t} \right\}_{t=1}^T, \text{ where } |\cdot| \text{ denotes the number of nodes in the set} \quad (6)$$

$$\lambda_i = \tanh(\text{MLP}(\text{BurstEmb}_i)), \text{ where } \text{BurstEmb}_i = \text{ReLU}(W_{burst} \text{NeighBurst}_i) \quad (7)$$

$$\text{BurstForecast}_i = \lambda_i * \text{Emv}_i \quad (8)$$

As a final remark, although our method adopts burstiness scores as input, it fundamentally differs from the burst detection work by Lappas et al. (2009). Their method takes a *retrospective, non-parametric* approach to identify burst segments within a *single, fully observed* time series. In contrast, our method is *prospective and parametric*, as we train a GNN module with *learnable parameters* to predict future bursts based on *multiple related time series*. More detailed comparison can be found in Appendix 4.

3.4. Shopping Journey Modeling via Attention-based Neighbor Embeddings

We apply Graph Transformer (Shi et al., 2021), an attention-based graph neural network (GNN) to learn the advance demand information hidden in customer shopping journeys. As discussed in Section 2.2, we need to estimate the conversions between the stages within a product or across products. With the clickstream graphs (see Figure 3), it is natural to use attention scores as a proxy for the conversion rates. Graph Transformer employs the flexible self-attention mechanism (Devlin et al., 2019) to learn complex associations between a focal node and its neighbors.

Specifically, we first assess how a source node time series (e.g., view) affects a target (e.g., purchase) using the self-attention mechanism. Each node embedding, say NodeEmb_i , is transformed through three dense layers to obtain three new representations denoted as query_i , key_i , and value_i (Equations 9-11). With node i as a central node, the edge weight ($W_{j,i}$) from its upstream neighbor j is also transformed by another dense layer to obtain an edge representation, $\text{edge}_{j,i}$ (Equations 12). Then, the attention from node i to node j ($\alpha_{i,j}$) is calculated as the exponential scale dot product between query_i and the sum of key_j and $\text{edge}_{j,i}$ (Equation 13). This score is normalized by Softmax across all upstream neighbors of i . Then we compute NeighEmb_i , a neighbor embedding of node i , as the attention-weighted sum of value_j of its upstream neighbors. Equations 9-14 describe the transformation by one-head attention. Multi-head attention can be achieved by duplicating this set of transformation to create multiple embeddings, which are concatenated to create a final neighbor embedding.

$$\text{query}_i = W_{\text{query}} \text{NodeEmb}_i \quad (9)$$

$$\text{key}_i = W_{\text{key}} \text{NodeEmb}_i \quad (10)$$

$$\text{value}_i = W_{\text{value}} \text{NodeEmb}_i \quad (11)$$

$$edge_{j,i} = W_{edge} W_{j,i} \quad (12)$$

$$\alpha_{i,j} = \frac{Att(query_{y_i}, key_{j+edge_{j,i}})}{\sum_{k \in N_i^{UP}} Att(query_{y_i}, key_{k+edge_{k,i}})}, \text{ where } Att(x_1, x_2) = \exp(\frac{x_1^T x_2}{\sqrt{d}}) \text{ is an exponential}$$

scale dot-product function and d is the dimension of x_1 or x_2 . (13)

$$NeighEmb_i = \sum_{k \in N_i^{UP}} \alpha_{i,k} value_k \quad (14)$$

Next, we use the neighbor embedding for forecasting. Each attention score, $\alpha_{i,j}$, serves as a proxy for the conversion rate from the upstream node j (e.g., cart of SKU A) to the central node i (e.g., purchase of SKU A). The larger the attention score, the more influence node j imposes on $NeighEmb_i$. Moreover, if node i 's neighbors include events of other products, e.g., view of SKU B, then inter-product transitions also contribute to $NeighEmb_i$. Similar to the base forecast described in Section 3.2., for node i , we generate another set of quantile forecasts, i.e., $NeighForecast_i \in \mathbb{R}^{H \times |Q|}$, from $NeighEmb_i$ using a new instantiation of the MLP network shown in Figure 2(B), i.e.

$$NeighForecast_i = MLP(NeighEmb_i) \quad (15)$$

At the last step, we adjust the $NeighForecast_i$ using a gate mechanism to reduce its overlap with the base forecast, $BaseForecast_i$ and burst forecasts $BurstForecast_i$. Customers who view a product or add a product to cart may place an order immediately. In this case, for node i , its neighbors' information is perhaps already reflected in its own time series embedding $NodeEmb_i$. In addition, in case of sales bursts, the neighbors' time series also contains burst information, which overlaps with $BurstEmb_i$. Adding $NeighForecast_i$ to $BaseForecast_i$ and $BurstForecast_i$ may inflate the forecasts. Thus, we use a gate mechanism to reduce the inflation. This gate concatenates three embeddings for node i : $NodeEmb_i$, $BurstEmb_i$, and $NeighEmb_i$, and uses a dense layer (with parameter W_{NeighW}) to collectively learn a vector of coefficients, β_i for each step within the forecast horizon (Equation 16). A sigmoid activation ensures $\beta_i \in \mathbb{R}^{H \times 1}$ is in the range of $[0,1]$, the percentages of $NeighForecast_i$ that can be taken into final forecasts (Equation 17). More elaboration of the gate mechanism can be found in Appendix 5.

$$\beta_i = \frac{1}{1+e^{-z_i}}, \text{ where } z_i = W_{NeighW} \text{Concat}(NodeEmb_i, BurstEmb_i, NeighEmb_i) \quad (16)$$

$$RegulatedNeighForecast_i = \beta_i * NeighForecast_i \quad (17)$$

3.5. Final Forecasts and Loss Function

Finally, we sum the three sets of forecasts as the final demand forecasts for node i at Q quantiles over a forecast horizon of H step, denoted as $Forecast_i \in \mathbb{R}^{H \times |Q|}$ (Equation 18). The quantile forecasts are compared with actual sales to calculate quantile loss (Wen et al., 2017). For each quantile q , let $Forecast_{i,T+h}^{(q)}$ denote the q -quantile forecast and at time $T+h$. The actual sales at time $T+h$ is $y_{i,T+h}$. The quantile loss is calculated using Equation (19):

$$Forecast_i = BaseForecast_i + BurstForecast_i + RegulatedNeighForecast_i \quad (18)$$

$$Loss_q \left(Forecast_{i,T+h}^{(q)}, y_{i,T+h} \right) = q \left(y_{i,T+h} - Forecast_{i,T+h}^{(q)} \right)_+ + (1 - q) \left(Forecast_{i,T+h}^{(q)} - y_{i,T+h} \right)_+ \quad (19)$$

where $(x)_+ = \max(0, x)$. For instance, let $q = 0.75, y_{i,T+h} = 10, Forecast_{i,T+h}^{(0.75)} = 8$, then $Loss_q(8, 10) = 0.75 * 2 + 0.25 * 0 = 1.5$. Finally, we sum the quantile loss across all quantiles, all forecast steps, and all target products to get the total loss (Equation 20).

$$Loss = \sum_i \sum_{1 \leq h \leq H} \sum_{q \in Q} Loss_q \left(Forecast_{i,T+h}^{(q)}, y_{i,T+h} \right) \quad (20)$$

4. Dataset and Evaluation

4.1 Dataset Description

We evaluate our model using a public clickstream dataset from Kaggle.³ This dataset contains 285 million user events collected from an e-commerce platform in the Middle East between October 2019 and April 2020. Because our model is designed to capture dynamic product relationships and detect early signals for sales bursts during promotional periods, we focus on sessions from the holiday season (October to December), when promotional activities are most frequent. This subset provides a natural and challenging testbed for our approach. It contains 38 million user sessions spanning over 239,718 SKUs. Each session records a series of events in an online shopping journey. An event is described by a timestamp, a product id, an event type (e.g., view), and other attributes such as selling price, category, and brand (see Figure 1(a)). Due to substantial missing values in these attributes, we exclude category and brand information in the main analysis and instead present a metadata-enhanced model in Appendix 6. We also ignore pricing variables as it is generally unrealistic to obtain detailed pricing schemes and promotion schedules in advance from sellers on marketplace or hybrid platforms (Tian et al., 2018).

To predict daily SKU sales, we construct a graph for each day using sessions from that day, as discussed in Section 2.2. Panel A of Table 2 shows the statistics of the daily graphs over the three months. On average, each graph consists of over 75,600 unique SKUs, 87,490 nodes, and 917,220 unique edges, with about 95% edges between different SKUs. These daily graphs vary substantially across days due to promotions or frequent SKU entry and exit. The largest graph has 40% more SKUs and six times more edges than the smallest graph.

With the graphs, we can investigate average conversions between the stages in the shopping journeys. Panel B of Table 2 shows the intra-product and inter-product transition probabilities. On average, after viewing an SKU, say A, the chance for the customer to view another SKU is about 50%, while the chance of adding SKU A to cart or purchasing it is only 3.58%. However, once adding SKU A to cart, the customer is likely to stick with it, with less than a 3% chance of switching to others. Wan et al.

³ <https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store/data>.

(2023) also found that customers are more likely to switch products in the early stage of the journeys than in the late stage. Finally, there is a 6% probability that a purchase event may lead to views of other products. This analysis indicates that both intra- and inter-product transitions are critical to sales forecasts.

Table 2: Clickstream Graph Summary Statistics

Panel A. <i>Daily Clickstream Graphs</i>							
Variable	Mean	Std	Min	25%	50%	75%	Max
Number of unique SKUs (thousand)	75.60	7.78	61.07	70.54	74.99	80.39	98.59
Number of nodes (thousand)	87.49	12.19	71.15	77.80	83.41	97.04	115.80
Number of unique edges (thousand)	917.22	627.74	377.25	433.25	510.91	1533.05	2365.07
- Intra-product edges	41.52	40.06	10.20	12.27	15.43	83.33	128.86
- Inter-product edges	875.70	589.17	366.45	420.51	498.33	1449.23	2236.22

Panel B. <i>Conversion between stages</i>							
Source \ Target	Intra-Product (%)			Inter-Product (%)			Terminate
	View	Cart	Purchase	View	Cart	Purchase	
View	22.58	3.58	3.58	49.73	0.02	0.00	23.73
Cart	38.24	14.95	28.88	2.97	0.02	0.00	14.94
Purchase	49.63	0.39	0.10	6.59	0.05	0.03	43.21

Panel C. <i>SKU intermittence within a month</i>							
Variable	Mean	Std	Min	25%	50%	75%	Max
Days with view events	14.15	10.62	0	4	13	24	31
Days with cart events	5.47	7.54	0	0	2	7	31
Days with purchase events	3.98	6.27	0	0	1	4	31

Panel D. <i>Daily event count per SKU (inactive days filtered)</i>							
View	21.81	185.95	1	2	4	11	58593
Cart	11.18	82.91	1	1	2	5	11148
Purchase (i.e., sales)	4.96	31.08	1	1	1	3	4469

Clickstream data is characterized by sparsity. As shown in Panel C of Table 2, within a month, 50% of SKUs have no purchase events. For each SKU, after filtering out the days without purchase events, as shown in Panel D, on average, there are about 5 purchase events, 11 cart events, and 22 view events per day. These counts are highly skewed. 75% SKUs have fewer than 3 purchase events, while a small portion of popular SKUs receive most attention from customers.

We implemented two strategies to tackle the data sparsity issue. First, to reduce noise, we use an edge threshold to remove incidental co-clicks. While a customer may click several products in a session, not all co-clicks reflect meaningful product associations. These incidental co-clicks typically correspond to infrequent edges in the graphs. We establish a co-click threshold, denoted as th_{edge} , to remove the edge between nodes i and j if it exists in fewer than th_{edge} number of daily graphs in the context window. For example, $th_{edge} = 3$ requires every edge should occur in at least three separate days (or three daily graphs). Second, we only target SKUs with frequent purchase events for model training and

evaluation. As indicated in Panel C of Table 2, 50% of products record sales on one day or fewer per month. Their sales time series are mostly filled with zeros. Forecasting models trained on such sparse data are likely to predict zeros by default. Therefore, it is meaningless to include these products in evaluating model performance. We select target products for sales forecasting as those with non-zero sales on at least 50% of the training days. This condition is met by 3445 SKUs in October and 3251 SKUs in November. The other SKUs remain in the graphs if they connect with the target SKUs. The forecasting of intermittent demand is out of the scope of this paper, because this requires a long sales history to learn the statistical demand distributions (Fildes et al., 2022).

We adopt a periodical model training/refresh strategy to ensure high accuracy in short-term (e.g., 3-5 days) daily sales forecasts. Short-term demand is highly sensitive to platform dynamics, such as product launches, discontinuations, or promotions. As shown in Table 2, Panel A, the largest daily sales graph contains 40% more SKUs and six times more edges than the smallest graph, indicating frequent SKU entry and exit. To maintain accuracy, the model must be regularly refreshed with recent data. Specifically, at the end of month t , the existing model (if any) is fine-tuned with data from month t , and then used to forecasts sales for month $t + 1$. With the holiday dataset, we implement a two-month sliding train-test cycle (see Figure 5): train on October and test on November, then fine-tune on November and test on December. In practice, the model can be updated more frequently (e.g., weekly) if needed.

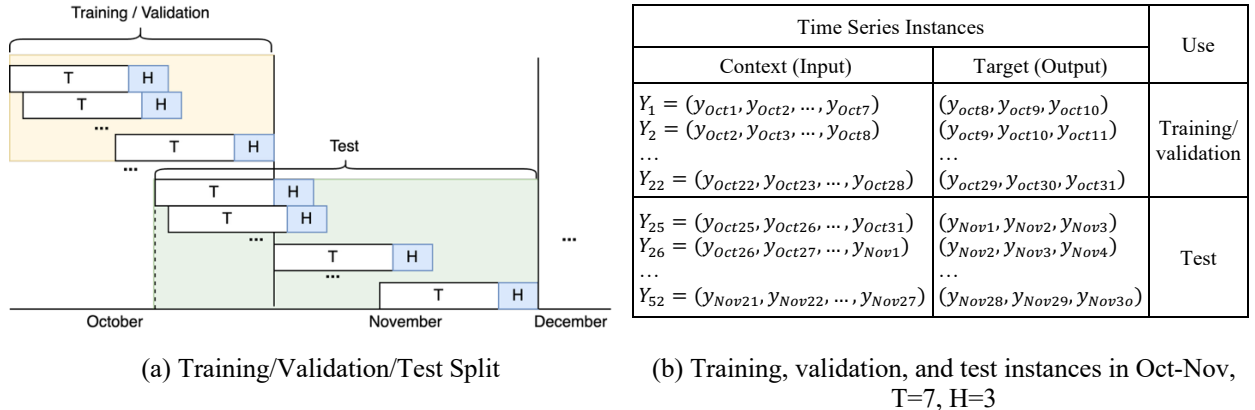


Figure 5: Time-based nested cross validation

We follow the time-based nested cross validation procedure (Sun et al., 2022) to split each train-test cycle into training, validation, and test time series subsets. For each cycle, we slice the sales history of each SKU into a collection of time series of length $T + H$, where $[1, T]$ is the context window and $[T + 1, T + H]$ is the prediction horizon. We start with $T = 7, H = 3$ and will perform a sensitivity analysis later. The test set includes time series with the prediction horizons falling into the second month. From the remaining data, time series from 10% of the SKUs are set aside for validation, while the rest are used for model training. To illustrate, Figure 5(a) shows the three subsets for the October-November cycle. Figure

5(b) illustrates the instances sliced from a two-month sales history of an SKU, denoted as $Y = (y_{Oct1}, y_{Oct2}, \dots, y_{Oct31}, y_{Nov1}, \dots, y_{Nov30})$. In total, for each SKU, there are 22 instances for training (or validation if the SKU is reserved for that purpose), and 28 test instances that involves November data. In each instance, the first seven values are model inputs and the last three serve as prediction targets.

4.2 Baseline Models, Evaluation Metrics, and Experiment Setup

Based on our discussion in section 2.4, we benchmark our proposed ForecastClickGraph with 14 widely adopted models from the literature, as shown in Table 3. The model implementation is described in Appendix 7. The clickstream dataset contains view, cart, and purchase event sequences within or across SKUs, which can be represented as covariates, time series, or graphs. However, not all models are able to leverage these features. For example, Prophet can only support univariate time series. Thus, we compare the model performance progressively under four scenarios:

- (1) *Univariate*: This group of models forecast an SKU’s sales only using its past sales data. Except for the graph-based models, we evaluate all the other models in this setup.
- (2) *Covariates*: Through feature engineering, we extracted daily view, cart, and purchase counts for each SKU as covariates, creating a multivariate time series per SKU as model inputs. Most univariate models, except Prophet, can be extended to include covariates.
- (3) *Graph-based*: Models leverage both intra-/inter-product transactions for sales forecasting. As illustrated in Figure 2(A), each model first generates a node embedding from the product’s time series through LSTM, and then applies graph convolution to incorporate neighbors’ information. This group includes three widely accepted models, GAT, Graph Transformer, and GraphSage.
- (4) *Dynamic GNNs*: Unlike the graph-based models that apply graph convolution only once, this group interweaves graph convolution into sequence learning (e.g., GRU or LSTM) by performing it at each time step. We include four models, T-GCN, GConvGRU, GConvLSTM, and GC-LSTM, in this group as they are representative in terms of both model structure and application domain. The detailed model comparison and selection justification can be found in Appendix 2.

We assess model performance using two widely adopted metrics: mean squared error (MSE) and symmetric mean absolute percentage error (SMAPE). MSE is biased toward large sales and is well-suited for scenarios where accurate sales forecasts for popular SKUs are crucial. Given M SKUs, MSE can be calculated by Equation (21), where $\hat{y}_{i,T+h}$, $y_{i,T+h}$ are median sales forecast and the true sales of SKU i at prediction horizon $T + h$, respectively. In contrast, SMAPE is a scale-free metric that equally weighs all SKUs. SMAPE can be calculated by Equation (22) and is ranged between $[0,1]$. Note that SMAPE is unreliable for extremely low-sale SKUs. For instance, if an SKU’s actual sales are zero, any non-zero forecast (e.g., 0.1 or 100), results in the maximum SMAPE of 1, regardless of the forecast error magnitude. For reliability, we calculate SMAPE only for SKUs with $y_{i,T+h} \geq 5$. It is also worth noting

that quantile loss optimization (Equations 19-20) drives the models to minimize forecast errors for SKUs with large sales. However, according to Equation (22), having large sales as the denominator always makes SMAPE improvements appear small. In addition, we train each model for 30 times each with a different random seed, and report the mean and standard deviations of these two metrics. We test the significance of model differences using Wilcoxon signed-rank test, as suggested by Steinker et al. (2017).

$$MSE = \frac{1}{M \cdot H} \sum_{i,h} (\hat{y}_{i,T+h} - y_{i,T+h})^2 \quad (21)$$

$$SMAPE = \frac{1}{M \cdot H} \sum_{i,h} \frac{|\hat{y}_{i,T+h} - y_{i,T+h}|}{(|\hat{y}_{i,T+h}| + |y_{i,T+h}|)} \quad (22)$$

Table 3: Benchmarking Models

Model	Description	Setting ^[1]
<i>Machine learning /Deep learning</i> ^[2]		
Prophet ^[3]	Prophet integrates three key components: trend, seasonality, and holiday effects learned through maximum likelihood estimation. Predictions are then sampled from the components.	U
MLP ^[4]	MLP maps an input time series into forecasts through multiple dense layers (see Figure 2(B)).	U, C
NHITS ^[5]	NHITS iteratively extracts unique information representing a characteristic (e.g., trending) of the input data through sampling and adds it to forecasts.	U, C
<i>Sequence Learning (Temporal)</i>		
LSTM ^[6]	See Section 3.2 for the details of LSTM	U, C, T
DeepAR ^[7]	DeepAR employs an auto-regressive RNN where the input consists of lagged target values and covariates. It assumes a probability distribution for the target values and estimates the distribution parameters during training.	U, T
Transformer ^[8]	Transformer embeds each element in a sequence through self-attention (see Section 3.4). It employs an encoder-decoder structure, where the encoder processes input time series and the decoder generates forecasts by referencing to encoder embeddings.	U, C, T
Informer ^[9]	Informer extends from Transformer by ProbSparse attention and distilling operations to focus on dominant information and to improve computational efficiency.	U, C, T
<i>Graph Neural Networks (Spatial)</i>		
Graph Transformer ^[10]	This model employs the self-attention to learn the attention scores between a focal node and its neighbors, and generates a new embedding for the focal node as the attention-weighted sum of the neighbors' information. See Section 3.4. for details.	C, G
GAT ^[11]	GAT is similar to Graph Transformer except that its attention mechanism is based on the similarity between two node embeddings.	C, G
GraphSage ^[12]	GraphSage transforms a node's features along with the aggregated information from its neighbors through a dense layer to generate a new node embedding.	C, G
<i>Dynamic GNNs (Spatial + temporal; see Appendix 2 for more details)</i>		
T-GCN ^[13]	TGCN, originally developed for traffic prediction, first applies graph convolution to extract spatial features at each snapshot, which are then fed into an RNN as sequential inputs.	C, T, G
GConvLSTM GConvGRU ^[14]	These two models embed graph convolution directly into the hidden state computations by modifying the LSTM/GRU architectures	C, T, G
GC-LSTM ^[15]	GCLSTM embeds graph convolution directly into the LSTM module, while also utilizing the network adjacency matrix within the LSTM to compute updated hidden states.	C, T, G
[1] Model evaluation settings, U: univariate, i.e., SKU purchase data only, C: covariate, i.e., SKU view, cart, and purchase as covariates, T: considering temporal dynamic in time series; G: considering neighborhood features from graphs (i.e., spatial) [2] We also trained ARIMA and SVR models for each SKU, but excluded them due to their poor performance. [3] Taylor and Letham, 2018; [4] Spiliotis et al., 2022; [5] Challu et al., 2023; [6] Hochreiter and Schmidhuber, 1997; [7] Salinas et al., 2020; [8] Devlin et al., 2019 [9] Zhou et al., 2021; [10] Shi et al., 2021; [11] Veličković et al., 2018; [12] Hamilton et al., 2017; [13] Zhao et al., 2020; [14] Seo et al., 2018; [15] Chen et al., 2022;		

5. Results

We compare our ForecastClickGraph learning framework with the benchmarking models across four scenarios (see Section 4.2): univariate, covariates, graph-based, and temporal graph-based to understand the effects of intra-product and inter-product transitions on sales forecasts. We also investigate how these models respond to sales bursts caused by promotions. Moreover, we evaluate the trained cross-learning models’ ability to predict sales of new products without retraining and discuss how to extend our model to non-holiday periods. In addition, we perform an ablation study to assess the effectiveness of our proposed modules and a sensitivity analysis on the impact of several key parameters.

5.1 Model Performance

Univariate group. As shown in Table 4, within the univariate group, Prophet performs the worst in both MSE and SMAPE, largely due to the data sparsity issue hindering model fitting for individual SKUs. In contrast, all the cross-learning models outperform Prophet by over 12% in MSE and 7% in SMAPE. Specifically, besides cross learning, MLP and NHITS benefit from their non-linear layers that can capture complex patterns within the time series. DeepAR slightly exceeds NHITS in MSE, but only achieves moderate performance due to the error propagation from its recursive forecast strategy and the challenge of fitting demands of all SKUs into a single probabilistic distribution. Transformer and its variant, Informer, achieve slightly lower MSEs than DeepAR, likely because the attention mechanism can effectively extract complex patterns from the input time series. LSTM coupled with quantile regression and direct multi-step forecasts achieved the best performance in this group. Interestingly, all the cross-learning models show similar results in the scale-free SMAPE metric. Models trained by quantile loss optimization are biased towards SKUs with large sales volumes, but minor improvements in forecast errors minimally influence SMAPE when the actual sales are large (see Equation 22).

Our model outperforms the univariate group in both MSE and SMAPE. Recall that our model integrates a univariate LSTM and graph-based modules for extracting features from intra-product and inter-product transitions. Compared to LSTM, our model reduces MSE by 15% and SMAPE by 3%. Univariate models, which rely solely on past sales history, overlook the advance demand information signaled by view and cart events of a product or interacting products.

Covariates. We evaluate three top-performing univariate models in a covariate setting that jointly incorporates view, cart, and purchase events. All covariate models have lower MSE and SMAPE than their univariate counterparts. In particular, NHITS reduces MSE by 6% due to NHITS’s incremental signal extraction from covariates. Our model still outperforms the covariate models by 12%-15% in MSE. One explanation is that covariate models capture only intra-product transitions and overlook advance demand signals embedded in inter-product relationships (e.g., co-view).

Table 4: Model Performance Benchmarking

Model	MSE			SMAPE (%) ^[4]		
	Mean	SD	Diff (%) ^[1]	Mean	SD	Diff (%) ^[1]
<i>A. Machine learning, deep learning, and sequence learning (Univariate)</i>						
Prophet ^[2]	596.60	-	28.25	35.68	-	10.09
MLP	528.87	12.86	19.06***	33.21	0.31	3.40***
NHITS	525.83	11.26	18.59***	33.38	0.50	3.89***
DeepAR	514.48	1.86	16.79***	33.36	0.29	3.84***
Informer	508.24	3.94	15.77***	33.22	0.73	3.43***
Transformer	506.14	1.69	15.42***	33.50	0.45	4.24***
LSTM	505.93	0.91	15.39***	33.09	0.11	3.05***
<i>B. deep learning and sequence learning (covariates; with SKUs' view and cart event as features) ^[3]</i>						
Informer	504.61	4.46	15.16***	32.81	0.77	2.22***
LSTM	497.38	2.37	13.93***	32.35	0.32	0.83**
NHITS	487.22	11.24	12.14***	32.23	0.65	0.47
<i>C. Graph Neural Networks (GNNs) (with aggregated neighbor event time series as features)</i>						
Graph Transformer	484.39	3.25	11.62***	33.38	0.31	3.89***
SAGE	482.94	2.48	11.36***	32.85	0.21	2.34***
GAT	480.60	4.18	10.93***	33.18	0.25	3.32***
<i>D. Dynamic GNNs (with aggregated neighbor events as features)</i>						
GC-LSTM	494.90	2.33	13.50***	32.63	0.18	1.69***
GConvGRU	492.63	1.86	13.11***	32.60	0.17	1.60***
GConvLSTM	493.76	2.21	13.31***	32.62	0.26	1.66***
T-GCN	473.84	1.96	9.67***	33.47	0.47	4.15***
Our Model	428.09	10.55	-	32.08	0.35	-
[1] *: $p < 0.10$, **: $p < 0.05$, ***: $p < 0.01$; one-sided Wilcoxon signed-rank test on the performance differences. [2] The Prophet model fitting is deterministic. Thus, its standard deviation is omitted. [3] Only top-5 univariate models are considered for the covariate setup. We skip DeepAR because it requires future view and purchase events as covariates for recursive forecasting. Transformer is ignored due to its similarity to Informer. [4] The differences in SMAPE are relatively small because models trained by quantile loss optimization focus on minimizing forecast errors for high-sales SKUs. According to Equation 22, when sales (i.e., $y_{i,T+h}$) are large, changes in SMAPE tend to be small.						

Graph-based. We continue to evaluate the effect of inter-product transitions using three popular graph neural networks (GNNs). As shown in Table 4, these models exhibit similar performance in MSE and SMAPE. Compared to the univariate and covariate models, they further reduce MSE and SMAPE by incorporating intra-/inter-product information through their graph structures. With the same input as the GNN baselines, our model outperforms them by about 11% in MSE and 3% in SMAPE. This performance gain highlights the advantages of our model architecture. Specifically, the weighted sum aggregation used by the baselines may overemphasize the advance demand information hidden in highly correlated view, cart, and purchase events, inflating the baseline forecasts. In contrast, our ForecastClickGraph model employs a gate mechanism to reduce the overlapping information.

Dynamic GNNs. We further evaluate advanced dynamic GNN models, T-GCN, GC-LSTM, GConvGRU, and GConvLSTM. These models integrate graph convolution with sequence learning by aggregating information from neighbors at every step of the time series. As shown in Table 4, they

perform slightly better than graph-based models in SMAPE but still fall short of our model, which outperforms them by 9–13% in MSE and up to 4% in SMAPE. The results suggest that while dynamic architectures can better capture time-varying patterns in neighbor events, they are still susceptible to noise and redundant signals. In contrast, our method relies on aggregated graphs and burstiness signals to reduce noise and a gate mechanism to mitigate redundancy.

Overall, our model consistently outperforms all baselines in terms of MSE and SMAPE. By comparing the models in the four scenarios, we demonstrate the effects of intra-/inter-product transitions on sales forecasts. Moreover, the high correlation among view/cart/purchase events necessitates selectively extracting advance demand information from them for accurate sales forecasts.

5.2 Burst Period Breakdown

Next, we analyze the model performance under active promotion periods. After sorting the total sales volume by date, we identified two time periods (11/14-20 and 12/15-28) with unusually high sales volume for most SKUs due to platform-wide grand promotions. To illustrate, Figure 6 displays the sales time series (grey lines) of two SKUs with these two periods highlighted. We can observe unusual sharp spikes in the figures. For instance, on December 16, SKU A’s sales reach 94, more than doubling the previous day’s sales. This spike is followed by a significant drop in sales, indicating reduced demand post-promotion as customers have stockpiled the SKU during the promotion (Hewage et al., 2022).

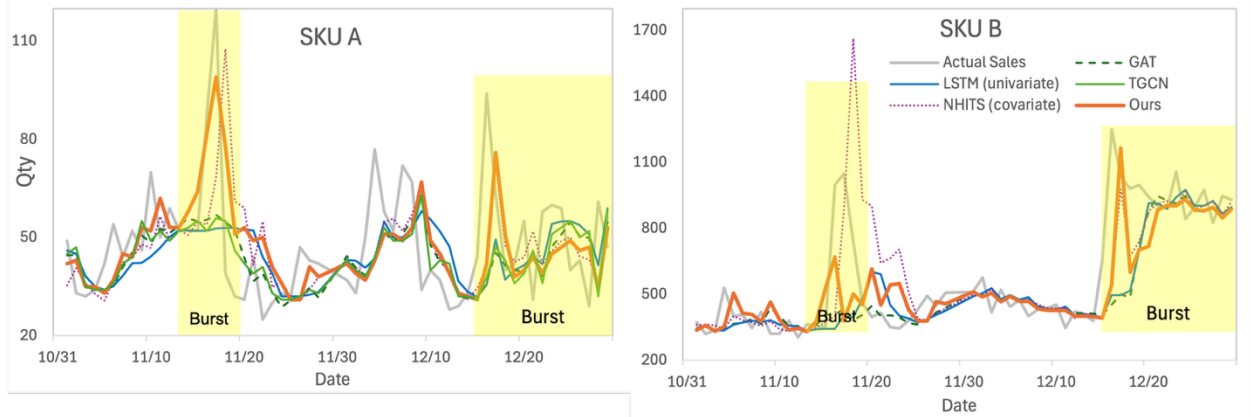


Figure 6: Actual and Forecasts of Example SKUs

During a holiday season, the success of an SKU largely hinges on the burst periods. Figures 7 (A-B) plot the total sales volume and amount in the burst periods (21 days in total) and the normal periods (40 days in total). Although the burst periods account for only one-third of the total holiday season, they contribute more than half (52%) of the total sales of the season. Thus, it is critical to have accurate SKU forecasts during the burst periods to support grand promotion activities. However, as shown in Figure 6, promotions can dramatically change customer behaviors and market dynamics, leading to irregular sales

spikes and drops that usually last only 1-2 days. The transient nature of sales bursts, combined with complex interactions among products, poses a significant challenge in accurately forecasting sales.

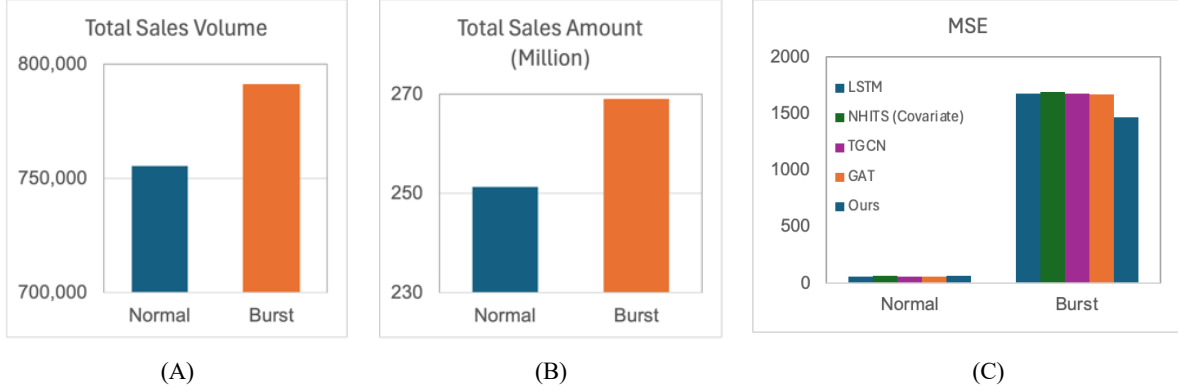


Figure 7: Sales and MSE in Burst and Normal Periods

Next, we compare model performance during burst and normal periods. We select four top-performing models across these groups in Table 4 and calculate its average MSE for the burst and normal periods separately. As shown in Figure 7 (C), the models have similar average MSE around 60 during the normal periods, but MSE during the burst periods exceeds 1400. As illustrated in Figure 6, sales during the normal periods are relatively stable, and even the univariate LSTM can accurately predict an SKU’s sales using only its sales history. In comparison, during the burst periods, the dramatic rise and fall in demand make sales forecasting challenging and result in large forecast errors. Among the models, our model has an MSE of 1466, while the best benchmark, GAT, reaches 1673, about 14% higher.

To illustrate, Figure 6 plots the forecasts from these four models. During promotions, our model timely detects dramatic demand changes and adjusts the forecasts in sync with the real demand. In comparison, the forecasts from other models either ignore the dramatic increase in demand or respond with latency. The delayed response results in even worse performance because a sharp increase is often followed by a substantial drop in sales. For example, the NHITS covariate model first underestimates the sales on the peak day (11/17) and then overestimates it on the following day. As a result, these models have large MSE around 1700 in the burst periods.

Our post-hoc analysis in Appendix 8 further shows that both graph-based modules play a critical role in forecasting during promotions. As shown in Equation (17), the neighbor context-based forecast, $RegulatedNeighForecast_i$, is regulated by an attention coefficient $\beta_i \in (0,1)$, while $BurstForecast$ is scaled by a burst attention ratio $\lambda_i \in (-1,1)$ (Equation 8). We extract the learned values of β_i and λ_i used in forecasting and average them across all products. The results show that β_i increases significantly during promotions, indicating that the model relies more heavily on neighbor context when generating forecasts. For example, during the major November promotion period, the average β_i increases from approximately 0.1 before the event to about 0.4 during the event. Similarly, the burst attention ratio λ_i is

predominantly positive during promotion periods, reflecting strong upward demand trends. Immediately afterward, however, λ_i often becomes negative, allowing the model to quickly adjust forecasts downward in response to reduced post-promotion demand and consumer stockpiling behavior (Hewage et al., 2022). Outside promotion periods, λ_i remains close to zero, indicating limited need for burst adjustment under stable demand conditions. The more details can be found in Appendix 8.

Overall, we showcased that our model is more responsive to dramatic demand changes caused by grand promotions than the state-of-the-art baseline models. As grand promotions can drive more than half of the sales during a holiday season, accurate forecasting is crucial for inventory and replenishment planning to ensure the successful execution of these promotions.

5.3 Forecasting New Product Demand (Cold-start)

By leveraging the generalized knowledge obtained from pooled product sales data, cross-learning models can forecast sales for new products not seen during model training. This capability is critical for e-commerce platforms with frequent product launches. To evaluate this capability, we selected the top-performing models from each group in Table 4. These models are trained on November sales data and directly used to forecast demands for products launched in December.

Table 5: Model Performance for New Product Sales Forecasting

Model ^[1]	Overall ^{[2] [3] [4]}				Burst Periods ^{[2] [3] [4]}			
	MSE	Diff (%)	SMAPE (%)	Diff (%)	MSE	Diff (%)	SMAPE (%)	Diff (%)
LSTM (Univariate)	6.22 (0.08)	16.08***	44.47 (0.05)	9.17***	6.65 (0.08)	17.59***	44.60 (0.05)	8.95***
NHITS (Covariate)	5.33 (0.14)	2.06	46.07 (2.59)	12.33***	5.57 (0.15)	1.62	47.93 (2.64)	15.27***
GAT (GNN)	6.13 (0.23)	14.85***	42.05 (1.8)	3.95**	6.51 (0.25)	15.82***	42.23 (1.78)	3.84**
T-GCN (Dynamic GNN)	6.16 (0.04)	15.26***	40.47 (1.03)	0.21	6.58	16.75***	40.88 (1.06)	0.65
Our Model	5.22 (0.36)	-	40.39 (0.77)	-	5.48 (0.41)	-	40.61 (0.75)	-
[1] Four top-performing models across groups in the previous Table 4 were chosen for this task. The models, trained on November sales, predict sales of products launched in December without retraining. [2] The standard deviations are shown in parentheses. [3] *: $p < 0.10$, **: $p < 0.05$, ***: $p < 0.01$; one-sided Wilcoxon signed-rank test on the performance differences. [4] SMAPE is calculated on SKUs with sales greater than 2.								

Understandably, new products have lower sales quantities and face a more severe data sparsity issue than established ones. The average daily purchase event count for new products is 1.41, compared to the overall average of 4.96 in the entire dataset (see Table 2, Panel D). Moreover, in the daily clickstream graphs, the average incoming degree of purchase nodes is 1.07, indicating that new products have few neighbors and cannot rely on graph-based features for forecasting. To address this issue, our evaluation

focuses on new products that have at least three days of non-zero view/cart/purchase events in the context window of length $T=7$. In total, 4769 new products are evaluated in this experiment.

As shown in Table 5, first, our model outperforms the others by up to 16% in MSE and 12% in SMAPE, with similar gains during burst periods. Second, among the benchmarking models, the covariate model NHITS exceeds the univariate model with a lower MSE, again highlighting the value of extracting demand signals from cart and view events. However, the graph-based GAT model performs worse than NHITS, likely due to sparse graph connectivity that hinders effective neighbor information extraction. The best temporal graph baseline T-GCN shows a similar pattern: although its SMAPE is on par with ours (40.47 vs. 40.39), its MSE remains significantly higher (6.16 vs. 5.22, +15.3%), indicating that coupling recurrence with graph convolution alone does not overcome this sparsity issue. In contrast, our model adopts a regulation gate to flexibly adjust its dependence on neighbors in case of reduced interconnectivity. Finally, compared to Table 4, new product MSEs are significantly smaller (5 vs. 428 for established ones), reflecting their typically low initial sales volumes.

5.4 Ablation Study and Sensitivity Analysis

Having demonstrated the effectiveness of our model, we now perform a thorough ablation study to understand the impact of our two task-specific design components. As discussed in Section 3, extending from the LSTM base, our model adds two graph-based components: burst prediction (Section 3.3) and neighbor embedding (Section 3.4.). Next, we remove each component one at a time to estimate its impact.

The ablation study presented in Table 6 underscores the importance of each component in our model. Removing the burst prediction module results in an 1.5% increase in MSE and 2.62% in SMAPE. Disabling the module for neighbor embedding leads to a 6.62% higher MSE and a 1.5% higher SMAPE overall. These ablation models demonstrate similar increases in MSE and SMAPE during burst periods as well. In particular, removing the burst prediction model increases MSE by 7.83%. Since both modules rely on neighbors’ information, when one module is removed, the other module may compensate to some extent. However, removing both graph-based modules results in a substantial MSE increase of over 18%.

Next, we analyze the sensitivity of the key parameters of our model as shown in Table 7. First, we assess the performance of our model under different forecast settings by changing the prediction horizon and the context window length. As expected, one-step prediction ($H = 1$) reduces MSE by about 20% and SMAPE by 28%, while five-step ahead prediction ($H = 5$) increases MSE and SMAPE by 18% and 19%, respectively. Overall, irregular time series challenge multi-step ahead forecasts because historical patterns may not apply to the future. However, a multi-step prediction horizon is often needed for estimating risks in inventory and replenishment decision making. We also find that changing the

context length deteriorates performance: longer contexts capture outdated trends, while shorter ones miss relevant information. For our dataset, a weekly context window is appropriate.

Table 6: Ablation Study

Setup	Overall ^{[1][2]}				Burst Periods ^{[1][2]}			
	MSE	Diff (%)	SMAPE (%)	Diff (%)	MSE	Diff (%)	SMAPE (%)	Diff (%)
Full	428.09 (10.55)	-	32.08 (0.35)	-	1465.68 (42.31)	-	42.59 (0.50)	-
Full – Burst Prediction	434.49 (9.13)	1.50**	32.92 (0.18)	2.62***	1580.38 (46.83)	7.83***	43.54 (0.23)	2.23***
Full – Neighbor Embedding	456.42 (6.62)	6.62***	32.56 (0.36)	1.50***	1496.52 (37.66)	2.10***	43.32 (0.52)	1.71***
Full – Burst Prediction – Neighbor Embedding	505.93 (0.91)	18.18***	33.09 (3.15)	3.15***	1757.63 (2.58)	19.92***	44.65 (0.13)	4.84***
[1] *: $p < 0.10$, **: $p < 0.05$, ***: $p < 0.01$; one-sided Wilcoxon signed-rank test on the performance differences. [2] The standard deviations are shown in parentheses.								

Second, we expand the neighbors of each target node from one tier to two tiers. As shown in Table 7, this slightly reduces MSE by 0.23% and SMAPE by 0.14%. This outcome can be explained by the example graph shown in Figure 3. The most influential neighbor of “A Purchase” is “A Cart”, a first-tier intra-product node with the highest transition probability (Panel B of Table 2). Its second-tier neighbors include all inter-product nodes, such as “B Cart,” and can only indirectly influence it through view nodes in the first tier. After two layers of propagation, the transition probability becomes even smaller. Thus, the impact of second-tier neighbors is very limited.

Table 7: Sensitivity Study

Setup		MSE ^[2]	Diff (%) ^[1]	SMAPE (%) ^[2]	Diff (%) ^[1]
Full Model ($th_{edge} = 3, \lambda = 0.7, L = 7, H = 3$, 1-tier neighbors)		428.09 (10.55)	-	32.08 (0.35)	-
Prediction Horizon (H)	1	343.48 (12.39)	-19.76***	23.12 (0.33)	-27.93***
	5	506.97 (9.93)	18.43***	38.37 (0.40)	19.61***
Context window length (L)	5	452.55 (11.27)	5.71***	32.14 (0.30)	0.19
	10	436.8 (10.11)	2.04***	32.63 (0.45)	1.71***
2-tier neighbors		427.10 (11.0)	-0.23	31.63 (0.36)	-0.14
Co-click threshold 4 (th_{edge})		428.36 (7.99)	0.06	32.17 (0.38)	0.28
Exponential Coefficient ($\lambda = 0.5$)		427.72 (10.25)	-0.09	32.10 (0.26)	0.06
[1] *: $p < 0.10$, **: $p < 0.05$, ***: $p < 0.01$; two-sided Wilcoxon signed rank test on the performance differences. [2] The standard deviations are shown in parentheses.					

In addition, we evaluate the sensitivity of two critical parameters of our model. As discussed in Section 4.1, the shape of the graphs is controlled by a co-click threshold (th_{edge}) to filter out incidental product interactions. For each node, its edges or co-clicks are tail-distributed, with only a few persistent neighbors indicating substitute or complement relationships. As shown in Table 7, increasing the threshold to 4 barely changes the model performance, indicating setting th_{edge} to a half of the window length seems appropriate. Finally, as described in Section 3.3, the burst prediction module produces a multiplier based on an exponential moving average with a coefficient λ . A smaller λ gives more weight to the early history of the time series. Reducing λ from 0.7 to 0.5 has minimal impact on performance.

5.5 Extension to Non-promotional Periods

Although our model is specifically designed to address forecasting challenges under promotional settings during holiday periods, it can be effectively extended for non-holiday periods. During holiday seasons, sudden sales bursts and dynamic product interactions are prevalent. Thus, a central component of our model is the burst detection module. In contrast, during the post-holiday period, large-scale joint promotions are relatively rare and sales patterns are more stable. To illustrate, Figure 8 shows the sales trajectories of two representative products from October to April. Compared with holiday season (October - December), sales variability during post-holiday period (January -April) is substantially lower, and burst events occur far less frequently. Across these four months, a noticeable burst is observed primarily around Valentine’s Day. This reduced frequency suggests that explicit burst modeling is not consistently required during non-holiday periods.

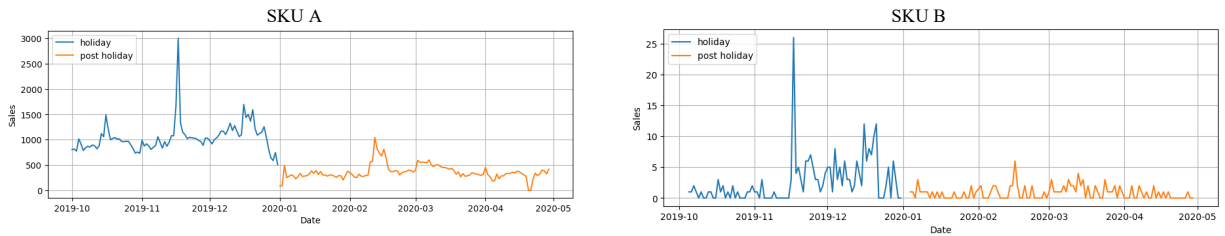


Figure 8: Sales of Two Example SKUs During Holiday and Post-holiday Periods

Thus, to adapt our model to non-holiday seasons, we introduce a gating mechanism that regulates the contribution of the burst detection module, as specified as in Equations (23-24). Specifically, the concatenated node, burst, and neighbor embeddings jointly learn a coefficient γ_i ($0 < \gamma_i < 1$) through a sigmoid activation function. The burst forecast is then multiplied by γ_i . When no burst behavior is detected, γ_i approaches to 0 and disables the burst detection module; while burst signals are present (e.g., Valentine’s Day promotion), γ_i moves to 1 and reactivates it.

$$\gamma_i = \frac{1}{1+e^{-z_i}}, \text{ where } z_i = W_\gamma \text{Concat}(\text{NodeEmb}_i, \text{BurstEmb}_i, \text{NeighEmb}_i) \quad (23)$$

$$\text{BurstForecast}_i = \gamma_i * \text{Emv}_i * \tanh(\text{MLP}(\text{BurstEmb}_i)) \quad (24)$$

We evaluate this modified model using sales data from January to April and benchmark it with the top-performing baseline models from each group (see Table 4). Following the same retraining strategy, each model is fine-tuned on the sales data from month t and generates forecasts for month $t + 1$. Model performance is assessed using MSE and SMAPE averaged over the four-month period. As shown in Table 8, our model outperforms these baselines with substantial improvements in MSE. Our model also outperforms the best dynamic GNN baseline by a clear margin. Notably, many products exhibit promotion-driven bursts around Valentine’s Day, and our model successfully captures these bursts, yielding lower forecast errors as measured by MSE. Since promotions are sparse during non-holiday seasons, all the models show comparable SMAPE when sales patterns remain relatively stable.

Table 8: Model Performance Benchmarking During Non-Holiday Seasons

Model	MSE	Diff (%)	SMAPE (%)	Diff (%)
LSTM (Univariate)	362.44 (2.24)	14.41***	30.27 (0.21)	0.17
NHITS (Covariate)	336.86 (16.31)	7.91***	31.02 (0.98)	2.58
GAT (GNN)	338.03 (2.88)	8.22***	30.97 (0.52)	2.42***
T-GCN (Dynamic GNN)	335.26 (2.44)	7.47***	33.94 (0.43)	10.96***
Our Model	310.23 (3.39)	-	30.22 (0.14)	-
[1] Four top-performing models across groups in Table 4 was chosen for this task. The burst detection module in our model was extended with a regulation gate as described in Equations (23-24).				
[2] The standard deviations are shown in parentheses.				
[3] *: $p < 0.10$, **: $p < 0.05$, ***: $p < 0.01$; one-sided Wilcoxon signed-rank test on the performance differences.				

5.6 Evaluation of Quantile Forecasts

While our analysis thus far has focused on point forecast accuracy (MSE and SMAPE), the value of our model also lies in its ability to produce well-calibrated quantile forecasts that enable retailers to achieve target service levels. In inventory management, service level (fill rate) refers to the probability of satisfying demand without stockouts (Liang et al., 2023). We first evaluate average quantile forecast performance using the quantile loss function introduced in Equations 19-20. This loss function penalizes under-forecasting more heavily for higher quantiles and over-forecasting more heavily for lower quantiles, which aligns with asymmetric costs in inventory management.

Table 9 reports the percentage improvements in quantile loss achieved by our model over top-performing temporal graph neural networks (T-GCN) and conventional models (LSTM, GAT) across the 50th, 75th, and 90th percentiles since the retail industry usually has a high service level. We specifically focus on the top three product categories (electronics, computers, and automotive) with sales observed over a 30-day period, as these categories account for the majority of total sales records. Consistent with

the importance of high service levels in retail operations, our model delivers the largest gains at the 90th percentile, which is critical for safety stock estimation, with improvements of 13.65% over LSTM, 12.65% over T-GCN, and 7.03% over GAT. At the 50th and 75th percentiles, our model also achieves substantial improvements (up to 7.25% and 11.53%, respectively). Averaged across the reported percentiles, our model achieves improvements ranging from 6.20% to 10.81% over the leading benchmarks. These results demonstrate that our model can support better inventory and fulfillment decisions across the mid-to-high demand range.

Table 9: Quantile Loss Improvement (%) by our Model at Each Percentile (Higher is Better)

Model	50% Quantile		75% Quantile		90% Quantile		Average Imp.
	Loss	Imp.	Loss	Imp.	Loss	Imp.	
Ours	22.55	---	24.40	---	21.45	---	---
LSTM	24.31	7.25	27.58	11.53	24.83	13.65	10.81
T-GCN	23.73	5.00	28.18	13.41	24.55	12.65	10.35
GAT	23.53	4.16	26.35	7.40	23.07	7.03	6.20

Beyond quantile loss, we assess how accurately each model achieves target service levels by analyzing forecast calibration. For a perfectly calibrated model, when setting inventory to the q -quantile forecast, the achieved service level should be exactly q . For example, if safety stock is set to the 90th percentile forecast, 90% of demand realizations should be met and stockouts should occur on no more than 10% of days during the experimental period. Figure 9 plots the target service level (nominal quantile) against the actual service level achieved (empirical coverage), where the perfectly calibrated forecasts would follow the diagonal line. While all benchmarks exhibit some mis-calibration, our model consistently tracks the diagonal more closely across all quantiles. The gap between our model and benchmarks is especially pronounced at high service levels (75–90%), which are critical for safety stock planning and service guarantees. For example, at 75th percentile, our model achieves a service level of 68.83%, 8.17% higher than the best benchmarking model. Overall, these results demonstrate that our model achieves not only better point forecast accuracy but also superior calibrated quantile forecasts.

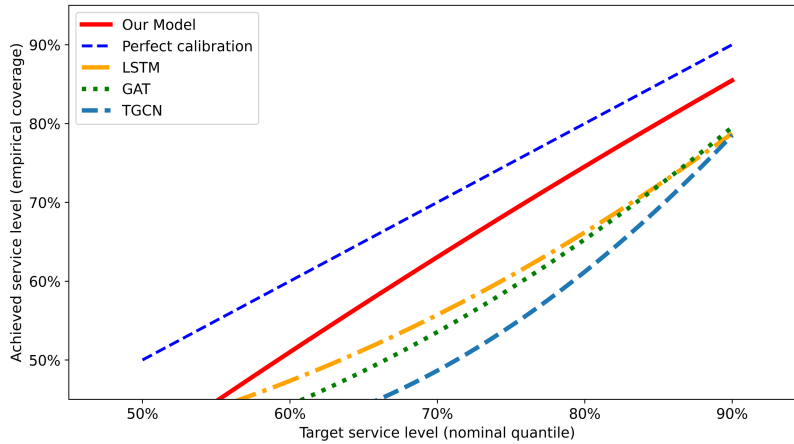


Figure 9. Forecast Calibration Analysis

6. Discussion and Conclusion

6.1 Summary of Findings

In this paper, we propose a deep learning framework for probabilistic demand forecasting using clickstream data from e-commerce platforms. This cross-learning framework incorporates two task-specific design choices to address challenges related to complex product relationships and promotional effects during holiday seasons. The underlying mechanism lies in our novel use of dynamic product relationships to capture sales patterns and anticipate changes in sales behavior. Specifically, our method includes two graph models. The first graph model explicitly captures how consumers transition across behavioral states (view/cart/purchase), as well as interactions among related products. This component models the normal sales dynamics, providing a baseline from which sales forecasts are generated. Promotions, however, introduce an “outlier” scenario in which sales behavior can deviate substantially from this norm. To account for this, the second graph model adjusts the baseline forecast by detecting promotion-driven anomalies. In particular, it identifies surges in product interactions as early signals of demand bursts (increased co-views), enabling timely correction of forecasts during promotion periods.

We tested this learning framework using a public clickstream dataset and benchmarked it against state-of-the-art forecasting models. Through extensive experiments, we demonstrated that our framework reduced MSE by up to 28% and SMAPE by up to 10%. Its burst prediction capability allows it to capture demand changes promptly, reducing MSE by more than 10% during active promotion periods. Moreover, as a cross-learning model, it effectively forecasts sales for new products without retraining. This model can also be readily extended to non-holiday periods by dynamically regulating the activation of the burst detection module when promotion-driven bursts are infrequent. This superiority is particularly evident in the quality of its probabilistic forecasts; the model delivers significant improvements in quantile loss, especially at the critical 75th and 90th percentiles, and provides better-calibrated predictions that more accurately achieve target service levels. These improvements are crucial for reliable inventory planning and risk management.

6.2 Research and Managerial Implications

This study extends the OM literature by advancing how clickstream data can be modeled for demand forecasting, while also offering actionable guidance for platform managers and sellers operating in decentralized marketplace environments.

Novel clickstream graph models: Our study introduces two novel graph models for clickstream data. The first model captures the collective flow of user attention across product stages (view, cart, and purchase). Unlike traditional approaches that assume static complementarity or substitution, our framework constructs directional graphs that capture dynamic dependencies across products (e.g., co-

view, co-purchase, view-after-purchase) along daily consumer purchase journeys. This representation allows us to capture early demand signals. For example, Figure 1 illustrates co-views among SKUs A, B, C, and D (smartphones and headphones) as a user intensively compares them before purchasing A and C. These co-view patterns therefore provide advance signals for forecasting demand for A and C. By embedding these product-stage graphs into graph neural networks, we translate granular behavioral interactions into structured forecasting signals at the SKU level. Second, we extend graph-based learning to the prediction of sharp and irregular demand spikes. We demonstrate that our model can proactively forecast sales bursts by learning from aggregated burstiness patterns within the product-stage graphs, while traditional time series models often treat sales bursts as “outliers” and fail to predict them. Together, these contributions demonstrate how clickstream data can be transformed from unstructured behavioral traces into a dynamic dependency structure suitable for sales forecasting.

Prioritizing product operations using graph connectivity: Our findings provide managers with a practical way to interpret cross-product demand dynamics using connectivity patterns derived from clickstream graphs. We draw on the distinction between search and experience products (Huang et al., 2009; Nelson, 1970) to interpret product dependencies. Search products (e.g., electronics and appliances) involve substantial pre-purchase comparison, generating dense connectivity. As a result, they often exert downstream influence on related products’ demand. In our empirical analysis, such products (e.g., SKU C in Figure 1) exhibit high connectivity, and the behavior of their neighboring nodes, such as co-views, contains valuable advance demand information that improves forecast accuracy. In contrast, experience products (e.g., personal care or fashion items) depend more on post-consumption learning and brand loyalty. These products display lower connectivity and more isolated demand dynamics, limiting their spillover potential. Therefore, our approach converts observed consumer search behavior into an interpretable dependency structure. Managers can use these connectivity patterns to identify leading products, refine operational plans, and strategically prioritize promotions on highly connected (search-type) products to stimulate broader demand spillovers.

Forecasting without full pricing transparency: Our study offers a practical demand forecasting solution for marketplace or intermediary platforms that often lack full access to proprietary pricing and promotion details (e.g., discount depth, promotion timing) from individual sellers. In many platform settings, pricing information is incomplete or inconsistently recorded. For example, in our dataset from a large intermediary platform, prices are not recorded for all events, and when present, they are inconsistently recorded due to diverse promotion mechanisms such as bundling, rebates, and personalized offers. To address this challenge, our paper focuses on an alternative and complementary source of demand signals: observed behavioral and sales dynamics. Customer browsing, comparison, and purchase transitions implicitly reflect the aggregate impact of heterogeneous and often unobserved promotion

strategies. By leveraging these behavioral signals, our approach captures the realized effects of seller actions without requiring explicit knowledge of pricing rules or future promotion schedules. This perspective is particularly valuable for platforms that do not directly control sellers’ pricing decisions and therefore cannot condition forecasts on future promotion schedules.

6.3. Boundary Conditions

Our method employs a dual-graph architecture to capture dynamic product relationships for sales forecasting. Specifically, the first graph models consumer transitions across behavioral states (view/cart/purchase) and interactions among related products, while the second graph detects surges in product interactions as early signals of demand bursts during promotion-periods. This design implies our proposed method is expected to perform well (1) when product interactions are rich and form highly connected graphs, enabling the graph-based learning of product relationships, and (2) when promotion events induce bursty sales patterns, allowing the burst-detection component to capture demand shocks that deviate from normal trends.

In contrast, our method may offer limited advantages over baseline models under three corresponding boundary conditions:

(1) *Sparse Connectivity*. Sparse graph connectivity reduces the usefulness of graph-based relationship modeling. To illustrate, we split products into high- and low-connectivity groups based on node degree, defined as the number of unique products connected to a given product, and evaluated forecasting performance separately for each group. We compared our model with two representative benchmarks: (a) LSTM, representing pure time-series approaches that ignore product interactions, and (b) Graph Attention Networks (GAT), representing state-of-the-art graph models. As shown in Table 10, in general, high connectivity indicates more user purchasing activities and higher sales volume, which in turn leads to larger MSE, compared with low-connectivity products. For the high-connectivity group, our model outperforms LSTM and GAT by margins over 12%. For the low-connectivity group, performance gains are understandably more modest, as limited neighborhood features constrain the benefits of our dual-graph design. These results confirm that rich interactions among products constitute an important boundary condition for the effectiveness of our approach. This boundary condition also limits the applicability of our method to highly intermittent products, whose infrequent sales typically cannot support a sufficiently connected graph over time. Accordingly, our claims of operational applicability apply primarily to active products with sufficient sales histories and graph connectivity.

Connectivity	MSE			Improvement (%)	
	Ours	LSTM	GAT	LSTM	GAT
High	1695.79	2024.10	1935.12	16.22	12.37
Low	10.89	11.16	11.02	2.42	1.18

Table 10: MSE Performance Comparison Across Connectivity Levels

(2) *Stable sales patterns without bursts*: The second boundary condition relates to the burst-detection component of our model. Our model is designed to capture promotion-driven sales bursts particularly during holiday or major campaign periods. Therefore, it may offer limited advantages when demand is relatively stable without surges. To illustrate, in Section 5.2 and Figure 7, we show that while our model substantially outperforms the baseline models during active promotion periods, it exhibits comparable performance to the baselines during non-promotion periods. To address this limitation, we introduce a gating mechanism that dynamically regulates the contribution of the burst-detection module (see Section 5.5).

(3) *Pricing and Meta Data Availability*: Our model is designed for intermediary or hybrid platforms where limited access to proprietary pricing and promotional information motivates reliance on observed behavioral and sales dynamics. However, in settings where pricing and promotion information (e.g., bundling rules) is accessible, as in reseller platforms (e.g., Bestbuy), it may be more efficient and accurate to foresee demand surges directly from these explicit features rather than from learned interactions.

6.4. Roadmap for Online Validation

Online controlled experiments (A/B tests) are the gold standard for quantifying the business value of algorithmic changes after deployment. For demand forecasting, the “treatment” is not a forecast shown to users but the operational decisions (e.g., replenishment, allocation, safety stock) derived from those forecasts. Therefore, a production-level A/B test should randomize the forecasting-to-decision pipeline at an appropriate operational unit (such as SKU clusters, categories, or fulfillment centers) to evaluate downstream KPIs like stockouts, fill rates, inventory turns, and service-level attainment. Because products interact through substitution and shared capacity, naive SKU-level randomization can induce interference; thus, clustered randomization or switchback designs are often necessary to mitigate spillovers (Kohavi et al., 2020, 2013). While our study relies on an anonymized public dataset and cannot execute a field experiment on the platform’s live system, we provide two critical practice-oriented anchors for future validation: (i) we evaluate probabilistic forecast calibration (Section 5.6), which maps directly to target service levels when inventory is set to specific quantiles, and (ii) we report performance under promotion-driven shifts and cold-start settings (Sections 5.2-5.3), which are the most operationally critical failure modes for retailers.

6.5. Limitations and Directions for Future Research

We also acknowledge several limitations of this study, thereby identifying future research opportunities. First, while our study is designed for settings where strategic pricing and promotion schedules are not readily accessible, incorporating such information when available could further enhance

model accuracy. Second, due to severe missing values in category and brand data in our dataset, our forecasting framework does not incorporate product metadata. Although our model can accommodate such features through embedding (Appendix 6), the dataset does not permit a more extensive exploration of metadata, which highlights an important direction for future research. Third, we chose aggregated daily demands as our forecast target, leaving the prediction of individual purchase behaviors for future work. Moreover, our method can be extended to predict other targets, such as shopping cart abandonment (Ding et al., 2015) or repurchase behavior. Finally, e-commerce platforms contain substantial numbers of seasonal and intermittent products, and future research could develop cross-learning methods that use information from related products to predict the onset, timing, and cessation of demand when an individual product has only a sparse sales history.

References

- Abbasi, A., Parsons, J., Pant, G., Sheng, O.R.L., Sarker, S., 2024. Pathways for Design Research on Artificial Intelligence. *Information Systems Research* 35, 441–459. <https://doi.org/10.1287/isre.2024.editorial.v35.n2>
- Abolghasemi, M., Beh, E., Tarr, G., Gerlach, R., 2020a. Demand forecasting in supply chain: The impact of demand volatility in the presence of promotion. *Computers & Industrial Engineering* 142, 106380. <https://doi.org/10.1016/j.cie.2020.106380>
- Abolghasemi, M., Hurley, J., Eshragh, A., Fahimnia, B., 2020b. Demand forecasting in the presence of systematic events: Cases in capturing sales promotions. *International Journal of Production Economics* 230, 107892. <https://doi.org/10.1016/j.ijpe.2020.107892>
- Adamopoulos, P., Todri, V., 2025. Consumer Social Connectedness and Persuasiveness of Collaborative-Filtering Recommender Systems: Evidence from an Online-to-Offline Recommendation App. *Production and Operations Management* 34, 4015–4038. <https://doi.org/10.1177/10591478241259422>
- Adomavicius, G., Tuzhilin, A., 2005. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE transactions on knowledge and data engineering* 17, 734–749.
- Baardman, L., Levin, I., Perakis, G., Singhvi, D., 2018. Leveraging Comparables for New Product Sales Forecasting. *Production and Operations Management* 27, 2340–2343. <https://doi.org/10.1111/poms.12963>
- Bai, L., Yao, L., Li, C., Wang, X., Wang, C., 2020. Adaptive graph convolutional recurrent network for traffic forecasting. *Advances in neural information processing systems* 33, 17804–17815.
- Bastani, H., Zhang, D.J., Zhang, H., 2022. Applied Machine Learning in Operations Management, in: Springer Series in Supply Chain Management, Springer Series in Supply Chain Management. Springer Nature, pp. 189–222. https://doi.org/10.1007/978-3-030-75729-8_7
- Besbes, O., Gur, Y., Zeevi, A., 2016. Optimization in Online Content Recommendation Services: Beyond Click-Through Rates. *M&SOM* 18, 15–33. <https://doi.org/10.1287/msom.2015.0548>
- Challu, C., Olivares, K.G., Oreshkin, B.N., Ramirez, F.G., Canseco, M.M., Dubrawski, A., 2023. NHITS: Neural Hierarchical Interpolation for Time Series Forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence* 37, 6989–6997. <https://doi.org/10.1609/aaai.v37i6.25854>
- Chen, G., Huang, L., Xiao, S., Zhang, C., Zhao, H., 2023. Attending to Customer Attention: A Novel Deep Learning Method for Leveraging Multimodal Online Reviews to Enhance Sales Prediction. *Information Systems Research* isre.2021.0292. <https://doi.org/10.1287/isre.2021.0292>

- Chen, J., Wang, X., Xu, X., 2022. GC-LSTM: graph convolution embedded LSTM for dynamic network link prediction. *Appl Intell* 52, 7513–7528. <https://doi.org/10.1007/s10489-021-02518-9>
- Chen, Z.-Y., Fan, Z.-P., Sun, M., 2023. Machine Learning Methods for Data-Driven Demand Estimation and Assortment Planning Considering Cross-Selling and Substitutions. *INFORMS Journal on Computing* 35, 158–177. <https://doi.org/10.1287/ijoc.2022.1251>
- Cheng, X., Deng, S., Jiang, X., Li, Y., 2023. Optimal promotion strategies of online marketplaces. *European Journal of Operational Research* 306, 1264–1278. <https://doi.org/10.1016/j.ejor.2022.08.020>
- Chi, Y., Lei, D., Zheng, L., Shen, Z.-J.M., 2024. Demand Forecasting During Grand Promotion for Online Retailing. Available at SSRN 4777632.
- CNBC, 2023. Three reasons a strong Black Friday weekend may not mean a blowout holiday season for retailers [WWW Document]. CNBC. URL <https://www.cnbc.com/2023/12/03/what-strong-black-friday-retail-sales-mean-for-holiday-spending.html> (accessed 6.4.24).
- Cohen, M.C., Zhang, R., Jiao, K., 2022. Data Aggregation and Demand Prediction. *Operations Research* 70, 2597–2618. <https://doi.org/10.1287/opre.2022.2301>
- Demirezen, E.M., Kumar, S., 2016. Optimization of Recommender Systems Based on Inventory. *Production and Operations Management* 25, 593–608. <https://doi.org/10.1111/poms.12540>
- Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, in: Burstein, J., Doran, C., Solorio, T. (Eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Presented at the NAACL-HLT 2019, Association for Computational Linguistics, Minneapolis, Minnesota, pp. 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- Ding, A.W., Li, S., Chatterjee, P., 2015. Learning User Real-Time Intent for Optimal Dynamic Web Page Transformation. *Information Systems Research* 26, 339–359. <https://doi.org/10.1287/isre.2015.0568>
- Ettl, M., Harsha, P., Papush, A., Perakis, G., 2020. A Data-Driven Approach to Personalized Bundle Pricing and Recommendation. *M&SOM* 22, 461–480. <https://doi.org/10.1287/msom.2018.0756>
- Feng, Q., Shanthikumar, J.G., 2018. How Research in Production and Operations Management May Evolve in the Era of Big Data. *Production and Operations Management* 27, 1670–1684. <https://doi.org/10.1111/poms.12836>
- Feng, Z., Wang, R., Wang, T., Song, M., Wu, S., He, S., 2025. A comprehensive survey of dynamic graph neural networks: Models, frameworks, benchmarks, experiments and challenges. *IEEE Transactions on Knowledge and Data Engineering* 38, 26–46. <https://doi.org/doi:%2010.1109/TKDE.2025.3621291>
- Ferreira, K.J., Parthasarathy, S., Sekar, S., 2022. Learning to Rank an Assortment of Products. *Management Science* 68, 1828–1848. <https://doi.org/10.1287/mnsc.2021.4130>
- Fildes, R., Goodwin, P., Önköl, D., 2019. Use and misuse of information in supply chain forecasting of promotion effects. *International Journal of Forecasting* 35, 144–156. <https://doi.org/10.1016/j.ijforecast.2017.12.006>
- Fildes, R., Ma, S., Kolassa, S., 2022. Retail forecasting: Research and practice. *International Journal of Forecasting* 38, 1283–1318.
- Fleder, D., Hosanagar, K., 2009. Blockbuster Culture’s Next Rise or Fall: The Impact of Recommender Systems on Sales Diversity. *Management Science* 55, 697–712. <https://doi.org/10.1287/mnsc.1080.0974>
- FRED, 2024. E-Commerce Retail Sales [WWW Document]. FRED, Federal Reserve Bank of St. Louis. URL <https://fred.stlouisfed.org/series/ECOMSA> (accessed 6.4.24).
- Goyal, P., Kamra, N., He, X., Liu, Y., 2018. DynGEM: Deep Embedding Method for Dynamic Graphs. <https://doi.org/10.48550/arXiv.1805.11273>
- Hagiu, A., Wright, J., 2015. Marketplace or Reseller? *Management Science* 61, 184–203. <https://doi.org/10.1287/mnsc.2014.2042>

- Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive Representation Learning on Large Graphs, in: Advances in Neural Information Processing Systems. Curran Associates, Inc.
- Hao, J., Zhao, T., Li, J., Dong, X.L., Faloutsos, C., Sun, Y., Wang, W., 2020. P-Companion: A Principled Framework for Diversified Complementary Product Recommendation, in: Proceedings of the 29th ACM International Conference on Information & Knowledge Management. Presented at the CIKM '20: The 29th ACM International Conference on Information and Knowledge Management, ACM, Virtual Event Ireland, pp. 2517–2524.
<https://doi.org/10.1145/3340531.3412732>
- Haruvy, E., Popkowski Leszczyc, P.T.L., Ma, Y., 2014. Does Higher Transparency Lead to More Search in Online Auctions? Production and Operations Management 23, 197–209.
<https://doi.org/10.1111/j.1937-5956.2012.01357.x>
- Hewage, H.C., Perera, H.N., De Baets, S., 2022. Forecast adjustments during post-promotional periods. European Journal of Operational Research 300, 461–472.
<https://doi.org/10.1016/j.ejor.2021.07.057>
- Hochreiter, S., Schmidhuber, J., 1997. Long Short-term Memory. Neural computation 9, 1735–1780.
<https://doi.org/10.1162/neco.1997.9.8.1735>
- Huang, P., Lurie, N.H., Mitra, S., 2009. Searching for Experience on the Web: An Empirical Examination of Consumer Behavior for Search and Experience Goods. Journal of Marketing 73, 55–69.
<https://doi.org/10.1509/jmkg.73.2.55>
- Huang, T., Fildes, R., Soopramanien, D., 2014. The value of competitive information in forecasting FMCG retail product sales and the variable selection problem. European Journal of Operational Research 237, 738–748. <https://doi.org/10.1016/j.ejor.2014.02.022>
- Huang, T., Van Mieghem, J.A., 2014. Clickstream Data and Inventory Management: Model and Empirical Analysis. Production and Operations Management 23, 333–347.
<https://doi.org/10.1111/poms.12046>
- Huang, T., Van Mieghem, J.A., 2013. The Promise of Strategic Customer Behavior: On the Value of Click Tracking. Production and Operations Management 22, 489–502.
<https://doi.org/10.1111/j.1937-5956.2012.01386.x>
- Jenkins, P., 2019. ClickGraph: Web Page Embedding using Clickstream Data for Multitask Learning, in: Companion Proceedings of The 2019 World Wide Web Conference. Presented at the WWW '19: The Web Conference, ACM, San Francisco USA, pp. 37–41.
<https://doi.org/10.1145/3308560.3314198>
- Jin, G., Liang, Y., Fang, Y., Shao, Z., Huang, J., Zhang, J., Zheng, Y., 2023. Spatio-temporal graph neural networks for predictive learning in urban computing: A survey. IEEE transactions on knowledge and data engineering 36, 5388–5408.
- Koenker, R., Bassett Jr, G., 1978. Regression quantiles. Econometrica: journal of the Econometric Society 33–50. <https://doi.org/10.2307/1913643>
- Kohavi, R., Deng, A., Frasca, B., Walker, T., Xu, Y., Pohlmann, N., 2013. Online controlled experiments at large scale, in: Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. Presented at the KDD' 13: The 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, Chicago Illinois USA, pp. 1168–1176. <https://doi.org/10.1145/2487575.2488217>
- Kohavi, R., Tang, D., Xu, Y., 2020. Trustworthy online controlled experiments: A practical guide to a/b testing. Cambridge University Press.
- Koren, Y., Bell, R., Volinsky, C., 2009. Matrix factorization techniques for recommender systems. Computer 42, 30–37.
- Kwark, Y., Lee, G.M., Pavlou, P.A., Qiu, L., 2021. On the Spillover Effects of Online Product Reviews on Purchases: Evidence from Clickstream Data. Information Systems Research 32, 895–913.
<https://doi.org/10.1287/isre.2021.0998>
- Lappas, T., Arai, B., Platakis, M., Kotsakos, D., Gunopulos, D., 2009. On burstiness-aware search for document sequences, in: Proceedings of the 15th ACM SIGKDD International Conference on

- Knowledge Discovery and Data Mining. Presented at the KDD09: The 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, Paris France, pp. 477–486. <https://doi.org/10.1145/1557019.1557075>
- Li, Y., Yu, R., Shahabi, C., Liu, Y., 2018. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. <https://doi.org/10.48550/arXiv.1707.01926>
- Liang, P., Cavusoglu, H., Hu, N., 2023. Customers' managerial expectations and suppliers' asymmetric cost management. *Production and Operations Management* 32, 1975–1993. <https://doi.org/10.1111/poms.13952>
- Long, X., Sun, J., Dai, H., Zhang, D., Zhang, J., Chen, Y., Hu, H., Zhao, B., 2025. The Choice Overload Effect in Online Recommender Systems. *M&SOM* 27, 249–268. <https://doi.org/10.1287/msom.2022.0659>
- Ma, S., Fildes, R., Huang, T., 2016. Demand forecasting with high dimensional data: The case of SKU retail sales forecasting with intra-and inter-category promotional information. *European Journal of Operational Research* 249, 245–257. <https://doi.org/10.1016/j.ejor.2015.08.029>
- Martínez-de-Albéniz, V., Planas, A., Nasini, S., 2020. Using Clickstream Data to Improve Flash Sales Effectiveness. *Production and Operations Management* 29, 2508–2531. <https://doi.org/10.1111/poms.13238>
- Nelson, P., 1970. Information and Consumer Behavior. *Journal of Political Economy* 78, 311–329. <https://doi.org/10.1086/259630>
- Olmezogullari, E., Aktas, M.S., 2022. Pattern2Vec: Representation of clickstream data sequences for learning user navigational behavior. *Concurrency and Computation* 34, e6546. <https://doi.org/10.1002/cpe.6546>
- Park, Y., Maddix, D., Aubet, F.-X., Kan, K., Gasthaus, J., Wang, Y., 2022. Learning quantile functions without quantile crossing for distribution-free time series forecasting, in: *International Conference on Artificial Intelligence and Statistics*. PMLR, pp. 8127–8150.
- Petropoulos, F., Apiletti, D., Assimakopoulos, V., Babai, M.Z., Barrow, D.K., Taieb, S.B., Bergmeir, C., Bessa, R.J., Bijak, J., Boylan, J.E., 2022. Forecasting: theory and practice. *International Journal of Forecasting* 38, 705–871. <https://doi.org/10.1016/j.ijforecast.2021.11.001>
- Qi, Yan, Li, C., Deng, H., Cai, M., Qi, Yunwei, Deng, Y., 2019. A Deep Neural Framework for Sales Forecasting in E-Commerce, in: *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. Presented at the CIKM '19: The 28th ACM International Conference on Information and Knowledge Management, ACM, Beijing China, pp. 299–308. <https://doi.org/10.1145/3357384.3357883>
- Rousseeuw, P.J., Croux, C., 1993. Alternatives to the Median Absolute Deviation. *Journal of the American Statistical Association* 88, 1273–1283. <https://doi.org/10.1080/01621459.1993.10476408>
- Sahni, N.S., Narayanan, S., Kalyanam, K., 2019. An Experimental Investigation of the Effects of Retargeted Advertising: The Role of Frequency and Timing. *Journal of Marketing Research* 56, 401–418. <https://doi.org/10.1177/0022243718813987>
- Salinas, D., Flunkert, V., Gasthaus, J., Januschowski, T., 2020. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International journal of forecasting* 36, 1181–1191. <https://doi.org/10.1016/j.ijforecast.2019.07.001>
- Schaer, O., Kourentzes, N., Fildes, R., 2022. Predictive competitive intelligence with prerelease online search traffic. *Production and Operations Management* 31, 3823–3839. <https://doi.org/10.1111/poms.13790>
- Semenoglou, A.-A., Spiliotis, E., Makridakis, S., Assimakopoulos, V., 2021. Investigating the accuracy of cross-learning time series forecasting methods. *International Journal of Forecasting* 37, 1072–1084. <https://doi.org/10.1016/j.ijforecast.2020.11.009>
- Seo, Y., Defferrard, M., Vandergheynst, P., Bresson, X., 2018. Structured Sequence Modeling with Graph Convolutional Recurrent Networks, in: Cheng, L., Leung, A.C.S., Ozawa, S. (Eds.),

- Neural Information Processing, Lecture Notes in Computer Science. Springer International Publishing, Cham, pp. 362–373. https://doi.org/10.1007/978-3-030-04167-0_33
- Shi, Y., Huang, Z., Feng, S., Zhong, H., Wang, W., Sun, Y., 2021. Masked Label Prediction: Unified Message Passing Model for Semi-Supervised Classification, in: Zhou, Z.-H. (Ed.), Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI 2021, Virtual Event / Montreal, Canada, 19-27 August 2021. [ijcai.org](https://doi.org/10.24963/IJCAI.2021/214), pp. 1548–1554. <https://doi.org/10.24963/IJCAI.2021/214>
- Shocker, A.D., Bayus, B.L., Kim, N., 2004. Product Complements and Substitutes in the Real World: The Relevance of “Other Products.” *Journal of Marketing* 68, 28–40. <https://doi.org/10.1509/jmkg.68.1.28.24032>
- Spiliotis, E., Makridakis, S., Semenovoglou, A.-A., Assimakopoulos, V., 2022. Comparison of statistical and machine learning methods for daily SKU demand forecasting. *Operational Research* 22, 3037–3061. <https://doi.org/10.1007/s12351-020-00605-2>
- Steinker, S., Hoberg, K., Thonemann, U.W., 2017. The Value of Weather Information for E-Commerce Operations. *Production and Operations Management* 26, 1854–1874. <https://doi.org/10.1111/poms.12721>
- Sun, C., Adamopoulos, P., Ghose, A., Luo, X., 2022. Predicting Stages in Omnichannel Path to Purchase: A Deep Learning Model. *Information Systems Research* 33, 429–445. <https://doi.org/10.1287/isre.2021.1071>
- Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., Jiang, P., 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer, in: Proceedings of the 28th ACM International Conference on Information and Knowledge Management. Presented at the CIKM '19: The 28th ACM International Conference on Information and Knowledge Management, ACM, Beijing China, pp. 1441–1450. <https://doi.org/10.1145/3357384.3357895>
- Taylor, S.J., Letham, B., 2018. Forecasting at Scale. *The American Statistician* 72, 37–45. <https://doi.org/10.1080/00031305.2017.1380080>
- Tian, L., Vakharia, A.J., Tan, Y. (Ricky), Xu, Y., 2018. Marketplace, Reseller, or Hybrid: Strategic Analysis of an Emerging E-Commerce Model. *Production and Operations Management* 27, 1595–1610. <https://doi.org/10.1111/poms.12885>
- Torry, H., 2020. Coronavirus Pandemic Widens Divide Between Online, Traditional Businesses. *Wall Street Journal*.
- Unal, M., Park, Y.-H., 2023. Fewer Clicks, More Purchases. *Management Science* 69, 7317–7334. <https://doi.org/10.1287/mnsc.2023.4716>
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y., 2018. Graph Attention Networks. Presented at the International Conference on Learning Representations.
- Voleti, S., Kopalle, P.K., Ghosh, P., 2015. An Interproduct Competition Model Incorporating Branding Hierarchy and Product Similarities Using Store-Level Data. *Management Science* 61, 2720–2738. <https://doi.org/10.1287/mnsc.2014.2039>
- Wan, X. (Shawn), Kumar, A., Li, X., 2025. Is It Beneficial to Recommend Differently Priced Products? Experimental Evidence From Online Recommender Systems. *Production and Operations Management* 10591478251392331. <https://doi.org/10.1177/10591478251392331>
- Wan, X. (Shawn), Kumar, A., Li, X., 2023. Retargeted Versus Generic Product Recommendations: When is it Valuable to Present Retargeted Recommendations? *Information Systems Research* [isre.2020.0560](https://doi.org/10.1287/isre.2020.0560). <https://doi.org/10.1287/isre.2020.0560>
- Wang, G., Zhang, X., Tang, S., Wilson, C., Zheng, H., Zhao, B.Y., 2017. Clickstream User Behavior Models. *ACM Trans. Web* 11, 1–37. <https://doi.org/10.1145/3068332>
- Wang, S., Hu, L., Wang, Y., He, X., Sheng, Q.Z., Orgun, M.A., Cao, L., Ricci, F., Yu, P.S., 2021. Graph Learning based Recommender Systems: A Review. <https://doi.org/10.48550/arXiv.2105.06339>
- Wen, R., Torkkola, K., Narayanaswamy, B., Madeka, D., 2017. A multi-horizon quantile recurrent forecaster. *arXiv preprint arXiv:1711.11053*.

- Wolters, J., Huchzermeier, A., 2021. Joint in-season and out-of-season promotion demand forecasting in a retail environment. *Journal of Retailing* 97, 726–745. <https://doi.org/10.1016/j.jretai.2021.01.003>
- Wu, S., Sun, F., Zhang, W., Xie, X., Cui, B., 2023. Graph Neural Networks in Recommender Systems: A Survey. *ACM Comput. Surv.* 55, 1–37. <https://doi.org/10.1145/3535101>
- Xue, G., Zhong, M., Li, J., Chen, J., Zhai, C., Kong, R., 2022. Dynamic network embedding survey. *Neurocomputing* 472, 212–223. <https://doi.org/10.1016/j.neucom.2021.03.138>
- Yan, A., Dong, C., Gao, Y., Fu, J., Zhao, T., Sun, Y., Mcauley, J., 2022. Personalized Complementary Product Recommendation, in: *Companion Proceedings of the Web Conference 2022*. Presented at the WWW '22: The ACM Web Conference 2022, ACM, Virtual Event, Lyon France, pp. 146–151. <https://doi.org/10.1145/3487553.3524222>
- Yan, S., Xiong, Y., Lin, D., 2018. Spatial temporal graph convolutional networks for skeleton-based action recognition, in: *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Yilmaz, Y., Öğüdücü, Ş.G., 2021. Enriching demand prediction with product relationship information using graph neural networks, in: *Proceedings of the 2021 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*. Presented at the ASONAM '21: International Conference on Advances in Social Networks Analysis and Mining, ACM, Virtual Event Netherlands, pp. 561–568. <https://doi.org/10.1145/3487351.3489477>
- Zhang, D.J., Dai, H., Dong, L., Qi, F., Zhang, N., Liu, X., Liu, Z., Yang, J., 2020. The Long-term and Spillover Effects of Price Promotions on Retailing Platforms: Evidence from a Large Randomized Experiment on Alibaba. *Management Science* 66, 2589–2609. <https://doi.org/10.1287/mnsc.2019.3316>
- Zhang, H., Wang, M., 2024. Tapping vs. Scrolling: Effects of Different Content Acquisition Modes on Content Consumption. *Inf Syst Front* 26, 1835–1855. <https://doi.org/10.1007/s10796-023-10429-y>
- Zhang, X., Zhao, Y., 2010. The Impact of External Demand Information on Parallel Supply Chains with Interacting Demand. *Production and Operations Management* 19, 463–479. <https://doi.org/10.1111/j.1937-5956.2009.01114.x>
- Zhao, L., Song, Y., Zhang, C., Liu, Y., Wang, P., Lin, T., Deng, M., Li, H., 2020. T-GCN: A Temporal Graph Convolutional Network for Traffic Prediction. *IEEE Transactions on Intelligent Transportation Systems* 21, 3848–3858. <https://doi.org/10.1109/TITS.2019.2935152>
- Zhou, H., Zhang, Shanghang, Peng, J., Zhang, Shuai, Li, J., Xiong, H., Zhang, W., 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting, in: *Proceedings of the AAAI Conference on Artificial Intelligence*. pp. 11106–11115. <https://doi.org/10.1609/aaai.v35i12.17325>
- Zhu, X., Ninh, A., Zhao, H., Liu, Z., 2021. Demand Forecasting with Supply-Chain Information and Machine Learning: Evidence in the Pharmaceutical Industry. *Production and Operations Management* 30, 3231–3252. <https://doi.org/10.1111/poms.13426>

Appendix 1: Modeling Clickstream Data by Graph Neural Networks: Literature Survey, Recommendation Systems, and Model Comparison

This appendix presents a survey of graph neural network (GNN) models applied to clickstream data, in particular, in recommendation systems. It also discusses the limitations of using these GNN models for sales forecasting and provides a detailed comparison between them and our proposed model.

A1-1. Literature Survey of GNN Models

To highlight the uniqueness of our graph model design and the necessity of our design, we conducted a survey of the use of GNNs in recommendation, online user behavior analysis, and forecasting, as shown in Table A1-1, with several example networks illustrated in Figure A1-1.

Study	Objectives	Graph Modeling		
		Node	Edge	Use of GNNs
[1] [2]	Recommending products to user by modeling or augmenting user-product, product sequences per user, user-user, and product knowledge graphs	User, product	User activity (e.g., purchase) between user-product pair (see Fig. A1-1(a))	Encoding user-product bipartite graphs for user and neighbor embeddings
		Product	A user's activity sequence (Fig. A1-1(b))	Product embeddings
		User	User social connections (Fig. A1-1(c))	User social network embedding
		Product	Product intrinsic relationships (Fig. A1-1(d))	Product embedding
[3]	Complementary product recommendation	Product	Inter-product co-purchase, co-view, and purchase-after-view based on clickstream data	Product embedding
[4]	Modeling user actions and predicting future user action (e.g., form fill), next URLs, and duration on a URL	Web pages	Links between web pages	URL embeddings by Node2Vec; embeddings are features in LSTM-based predictions.
[5]	Clustering user behavior based on sequences of click events to capture distinct user groups for the detection of malicious accounts	User activities (e.g., posting, chatting)	Transition between online activities	Clustering graphs based on common subsequences; classification by features extracted from online activities
[6]	Clustering user navigation paths to discover common patterns for UI testing	URLs	Clicks between pages	Embedding URLs and user navigation paths as weighted sum of URL embeddings
[7]	Augmenting demand forecasting using production relationships	Product	Product associations based on the sales of products in the same week, co-purchase, or bought by the same customer	Embedding products by node2vec
Ours	Demand forecast based on inter-/intra product stage transitions and neighbor burstiness	Product + Stage, time series attributes	Aggregated intra-/inter product stage transitions during customer shopping journeys	Dynamic neighbor embedding and neighbor burstiness embedding
[1] Wu et al., 2023; [2] Wang et al., 2021; [3] Hao et al., 2020; [4] Jenkins, 2019; [5] G. Wang et al., 2017; [6] Olmezogullari & Aktas, 2022; [7] Yilmaz & Öğüdücü, 2021				

Table A1-1: Survey of GNN models Constructed from Clickstream

Modeling item interrelationships using graphs is a well-established approach. In recommendation systems, graph models are typically used to represent user-product interactions, product-product sequences, user social networks, or product knowledge, as illustrated in Figures A1-1(A-D) (Wang et al., 2021; Wu et al., 2023). Another body of literature uses graphs to capture the interconnectivity of web pages (Jenkins, 2019), or the sequences of user actions (Olmezogullari and Aktas, 2022; Wang et al., 2017). Several notable studies take a data-driven approach to extract product relationships from clickstream data and modeling them as graphs (Hao et al., 2020; Yilmaz and Öğüdücü, 2021). For example, Figure A1-1(E) shows a graph that capture inter-product relationships such as co-view and co-purchase. These relationships are learned offline and subsequently used to enhance recommendation or forecasting tasks.

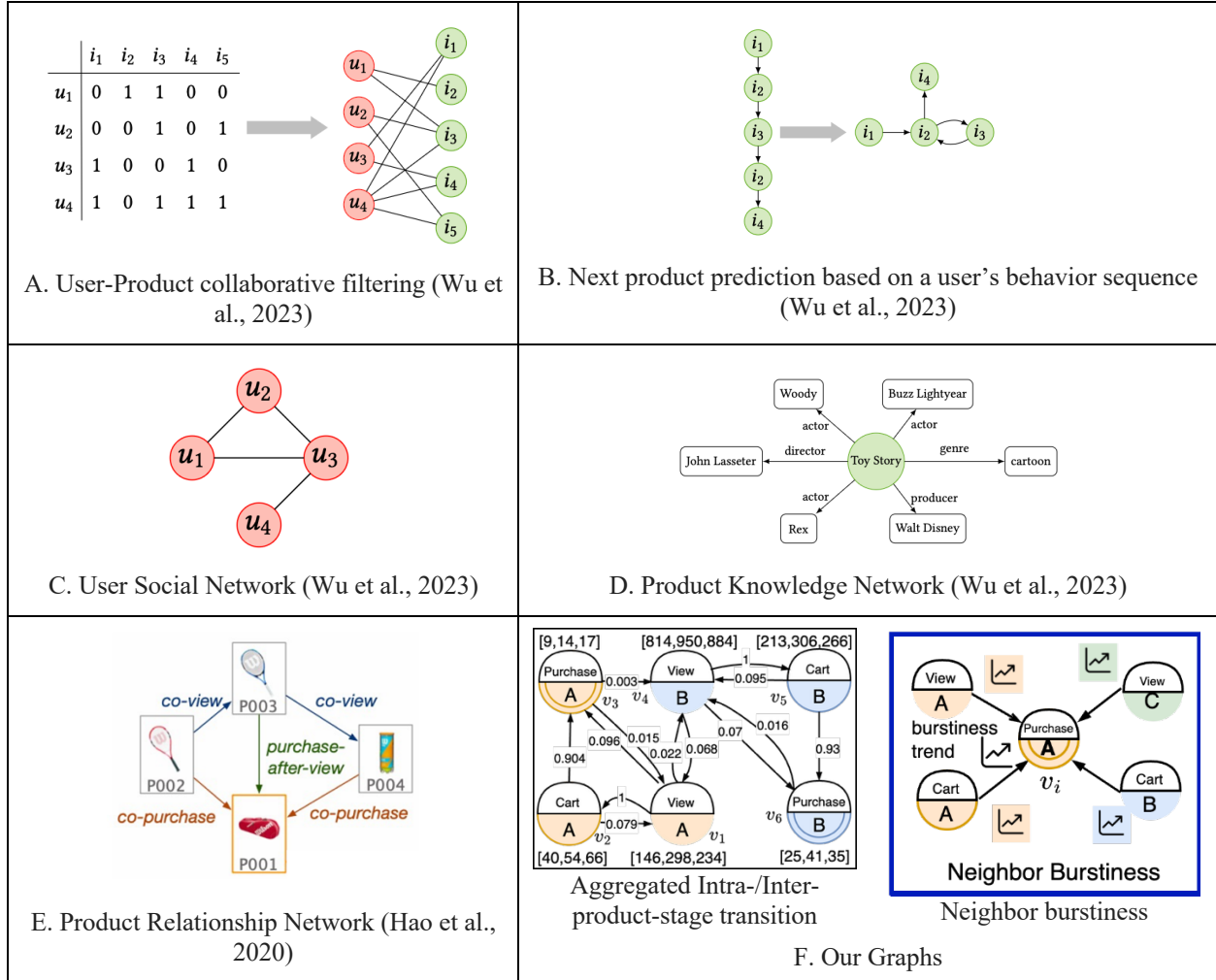


Figure A1-1: Example Graph Models

A1-2. Recommendation vs. Sales Forecasting

Recommendation systems often leverage GNNs to model individual user behaviors (e.g., Figure A1-1 (A-C)). While state-of-the-art recommender models, such as Transformer-based BERT4Rec (Sun et al., 2019) or graph-based LightGCN (He et al., 2020), excel at ranking items to optimize user engagement, Table A1-2 highlights that sales forecasting serves a fundamentally different operational objective. The question is whether we can forecast sales based on individual user behaviors (e.g., Sun et al., 2022; Wan et al., 2023). An intuitive idea is that if we can estimate the probability of a consumer purchasing a

product given their current shopping journey and shopping history, then we can aggregate these probabilities to obtain expected sales forecasts.

Aspect	Recommendation systems	Sales forecasting (this paper)
Primary Objective	Personalize/rank items to influence user choices and conversion	Plan supply decisions by predicting future realized demand
Key Literature	BERT4Rec (Sun et al., 2019); SR-GNN (Wu et al., 2019); LightGCN (He et al., 2020)	Demand forecasting (e.g., Salinas et al., 2020; forecasting with clickstream: Huang and Van Mieghem, 2014)
Unit of Prediction	User–item interaction / session next-item / ranked list	SKU-level demand time series aggregated across users
Inputs	User history + item features; user–item or session graphs	Sales history + aggregated product-stage transition graphs (views/carts/purchases)
Output	Interaction probability / top-K recommendations	Multi-step probabilistic demand forecasts (quantiles)
Typical Metrics	HR@K, NDCG@K, AUC, CTR	MSE, SMAPE, Quantile Loss, Service Level Calibration

Table A1-2: Key distinctions between Recommendation and Sales Forecasting

However, this approach requires a few additional assumptions, as reasoned below:

- First, sales forecasting fundamentally aims to predict the *total number of units sold*. This total volume is a function of two key components: (a) the *number* of potential customers arriving or initiating interest in a product/category, and (b) the *probability* that an arriving customer converts to a purchase. Recommendation systems, in contrast, primarily focus on component (b) – optimizing the conversion probability or predicting an interaction for a user who is already present or whose context is known. Indeed, studies on individual shopping journeys often focus on the stages after a consumer shows interest in a product, e.g., clicking it (Wan et al., 2023) or on the purchase behaviors of existing customers (Sun et al., 2022). They typically do not aim to predict the initial volume of customer arrivals to the platform or to specific product areas. Therefore, a critical distinction is that sales forecasting must model or account for the customer arrival process, which is a complex task in itself (especially for thousands of SKUs with varying popularity, new customer influx, and sparse interactions). This is generally outside the primary scope of recommendation models, which often operate conditional on user presence or an initiated interaction.
- Second, sales forecasting needs to predict how many sales will occur within a specific future horizon (e.g., daily sales for the next H days). Recommendation models often predict the *likelihood* of a future interaction but not precisely *when* it will occur. Estimating the conversion timing requires modeling factors beyond user preference, such as urgency, promotions, and external events. These factors add complexity not inherent in typical recommendation tasks.
- Moreover, aggregated sales are heavily influenced by factors like promotions, seasonality, and emergent “herd behaviors” (Chi et al., 2024), especially during holidays. These aggregate dynamics and externalities are central to sales forecasting but often peripheral to models focused on individual user preference prediction. Our burst prediction module, for example, explicitly targets these aggregate phenomena.

In summary, while GNNs are versatile, their application in sales forecasting involves a novel graph representation specifically engineered for aggregate forecasting dynamics and includes forecasting-specific architectural components (like burst prediction) not found in typical recommendation systems. Furthermore, fundamental differences in objectives (aggregate volume vs. individual likelihood) and necessary inputs (explicit modeling of arrival processes, timing considerations, aggregate effects vs. user

history) make direct adoption of recommendation approaches infeasible for achieving accurate, multi-step probabilistic sales forecasts in a large-scale e-commerce setting.

A1-3. Comparing our Model with Existing GNN approaches

As summarized in Table A1-1, our graph model, shown in Figure A1-1(F), differs significantly from existing models in the literature in several key aspects:

- **Nodes and node attributes:** In prior work, products are typically modeled as nodes with static node information (e.g., name, quality). In contrast, our model defines each node as a *product-stage* pair (e.g., view of SKU A). To capture the temporal dynamics of customer behaviors, each node is associated with two distinct type of time series:
 - *Quantity time series*, which captures aggregated customer activities and its evolution over time.
 - *Burstiness time series*, which captures sudden spikes or bursts in customer behaviors, and is derived from the node time series (see Equations 5-6).
- **Edges:** While existing models focus on a limited set of manually selected inter-product relationships, e.g., co-view, co-purchase (Figures A1-1(B, E)), or user-product interactions (Figure A1-1(A)), our graph extends the concept of edges to serve three distinct purposes:
 - *Intra-product transitions.* These edges model the customer journey within a product. Beyond standard transitions such as *view* $\rightarrow$ *cart* $\rightarrow$ *purchase*, our approach can automatically identify context-specific patterns that are significant, such as purchase-after-view (e.g., one-click buy; Unal & Park, 2023), or purchase-after-purchase (i.e., repeated purchases).
 - *Inter-product transitions:* While prior work (e.g., Hao et al., 2020) focused on co-view, co-purchase, view-after-purchase relationships, our general framework can flexibly capture a broad range of nuanced inter-product relationships. For example, Zhang et al. (2020) observed shopping cart promotions led to an increase in co-cart behaviors—customers adding multiple products to their cart. In our graph-based framework, this relationship is naturally represented by edges between cart nodes. Another example involves customers adding several products to their cart but ultimately purchasing only one, perhaps due to perceived quality, price elasticity, response to promotions, consumption context (Voleti et al., 2015). Such complex patterns can be modeled through the analysis of co-cart and cart-to-purchase transitions within our framework.
 - *Upstream/downstream neighbors:* As described in Section 3.1, These intra-/inter-product transitions naturally lead to the construction of neighbor context, which we leverage for identifying neighbor burstiness.
- **Dynamic graphs:** Unlike the static graphs commonly used in prior literature, our framework constructs dynamic graphs by aggregating clickstream data within specific time windows (e.g., daily). This approach allows the model to capture real-time changes in customer behaviors, which is especially critical during active promotion periods. For example, Figure 3 highlights how edges and node attributes are continuously updated based on user activity.

To summarize, our framework adapts the design of nodes, edges, and graph construction to effectively capture:

- Temporal dynamics of customer behaviors
- Neighbor burstiness
- Complex intra-product transition patterns in customer shopping journeys
- A broad range of nuanced inter-product relationships

These adaptations are essential for addressing the challenges in forecasting sales during active promotions, as customer behaviors and product relationships deviate dramatically from typical patterns. These deep domain adaptations present a pathway for a deep-learning based design science artifact (Abbasi et al., 2024).

Appendix 2: Comparing our Model with Dynamic Graph Neural Networks

In this appendix, we first survey dynamic graph neural networks (GNN) models, and then compare our model with several representative dynamic GNN baselines to highlight the necessity of our customized design for domain adaptations in sales forecasting.

(1) Survey of Dynamic GNNs and the selection of baseline models

Dynamic GNNs have been used to study dynamic networks such as social networks and traffic networks (Feng et al., 2025; Xue et al., 2022). A special class of dynamic GNNs, called *spatio-temporal* models, often assume a fixed node set with time-varying node features or signals (Feng et al., 2025; Jin et al., 2023). In general, these models combine a spatial operator for capturing relationships among connected nodes with another temporal operator for capturing changes over time. The main differences lie in the specific type of graph or temporal operator used. Table A2-1 lists eight dynamic GNN models with their specific spatial and temporal operators.

Model	Spatial operator	Temporal operator	Primary application	Included in our study
T-GCN [1]	Graph convolution (GCN) over a predefined graph	GRU	Traffic forecasting	Yes
GConvGRU GConvLSTM [2]	Graph CNN using spectral graph convolution	GRU LSTM	Video classification and NLP part-of-speech (POS) tagging	Yes
GC-LSTM [3]	GCN	LSTM	Dynamic network link prediction	Yes
ST-GCN [4]	GCN over predefined skeleton network	Temporal convolution (TCN)	Video classification: skeleton-based action recognition	No
ST-GAT [5]	Graph attention over spatial neighbors	LSTM	Traffic forecasting	No
DCRNN [6]	Diffusion convolution based on bidirectional random walks	GRU	Traffic forecasting	No
AGCRN [7]	Adaptive GCN based on learned node embeddings and latent adjacency	GRU	Traffic forecasting with unknown or incomplete edges in graphs	No
[1] (Zhao et al., 2020) [2] (Seo et al., 2018) [3] (Chen et al., 2022) [4] (Yan et al., 2018) [5] (Kong et al., 2020) [6] 9/29/26 8:25:00 PM [7] (Bai et al., 2020)				

Table A2-1: Comparing Dynamic GNN Models

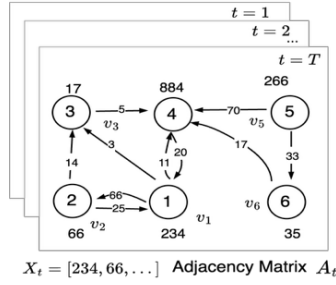
According to Xue et al. (2022), dynamic GNN methodologies can be broadly categorized into two groups: *structural first* or *temporal first*.

The structural-first category focuses on learning the underlying structure of dynamic networks and incrementally updating it over time. For example, DynGEM (Goyal et al., 2018) uses an autoencoder to incrementally learn embeddings of dynamic networks. Specifically, it initializes the embedding at time t with the learned embedding from time $t - 1$, and then continues the model training on the updated network. Approaches in this category typically assume the network structures are relatively stable, and their learning objectives often depends on structural properties, such as community clustering and anomaly detection.

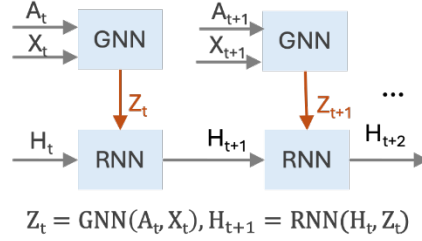
In contrast, the temporal-first category incorporates network structural features at each time step into a sequence learning framework. A representative model in this category is T-GCN (Zhao et al., 2020),

shown in Figure A2-1(B), which was developed for traffic prediction. Given a series of graphs $G_1, G_2, \dots, G_T$, where each graph at time t , denoted as $G_t = (A_t, X_t)$, consists of an adjacency matrix A_t and node features X_t (Figure A2-1(A)), T-GCN first uses a GNN module to extract the neighborhood information for each node. The output of the GNN at time t , denoted as Z_t , is then passed to an RNN module for the temporal modeling.

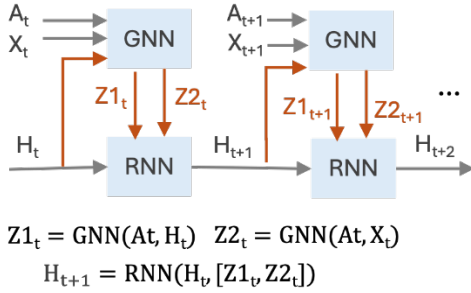
In general, temporal-first models differ in how the GNN and RNN components are integrated. Figures A2-1(B-D) illustrate three representative models, each with a distinct integration pattern. T-GCN connects the GNN and RNN modules in a unidirectional manner, where the GNN output serves as input to the RNN. In contrast, the other two models adopt a more reciprocal integration. In the GConvGRU/GConvLSTM models (Seo et al., 2018) (Figure A2-1(C)), the hidden states of the RNN are fed into a GNN module as node features, while a separate GNN module performs standard graph convolution on the node features X_t . This design enables the model to effectively capture spatial-temporal dependencies in video frames. Similarly, GC-LSTM (Chen et al., 2022) (Figure A2-1(D)) incorporates RNN hidden states into graph convolution, while also utilizing the adjacency matrix A_t within the RNN to compute updated hidden states. This bidirectional integration has shown to be effective for tasks like link predictions in recommendation systems.



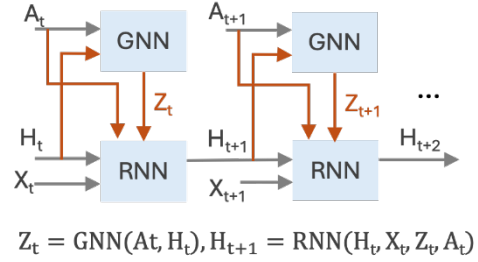
(A) Input Graphs



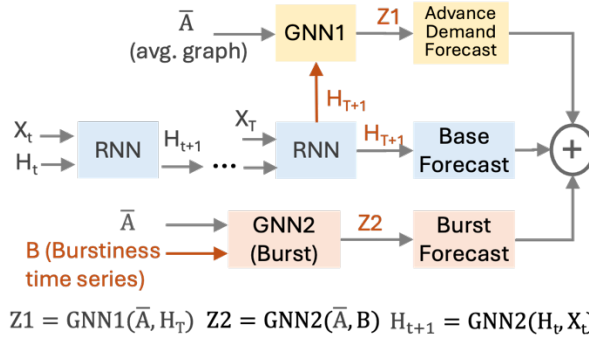
(B) T-GCN (Zhao et al., 2020)



(C) GConvGRU/GConvLSTM (Seo et al., 2018)



(D) GC-LSTM (Chen et al., 2022)



(E) Ours (a rearranged version of Figure 2 from the manuscript)

Figure A2-1: Comparing our Model with Dynamic GNNs

We selected these representative temporal-first models shown as Fig. A2-1(B-D) for benchmarking because they are well aligned with our sales forecasting context. First, structural-first models typically emphasize learning persistent network structures, whereas our focus is on capturing the temporal dynamics of nodes. According to forecast literature, capturing the temporal dynamic in the time series sales history of SKUs is critical for accurate sales forecasting (Fildes et al., 2019). Second, in our setting, networks derived from customer shopping journeys exhibit substantial variability due to promotions and frequent SKU entry and exit. For instance, as shown in the Panel A of Table 2 in the manuscript, the largest graph has 40% more SKUs and six times more edges than the smallest graph. This high degree of structural variability limits the effectiveness of spatio-temporal models that rely on persistent structural features. Third, some alternative models often rely on assumptions that are less well aligned with our setting. For instance, DCRNN (Li et al., 2018) was designed for traffic networks, where movement are modeled as physical flow along roads. Our graph instead represents consumer actions and transitions among products and shopping stages, which do not follow the same diffusion process. Similarly, AGCRN (Bai et al., 2020) is designed to learn an unknown graph from the data, whereas our clickstream graph is directly observed and is itself an important source of behavioral information. Replacing this observed graph with a learned latent graph could remove the consumer-transition information that our model is designed to exploit.

In addition, the selected baseline models are representative in terms of both model structure and application domain. As shown in Table A1-1, dynamic GNN models mainly differ in their graph or temporal operators. Collectively, our selected baseline models cover commonly used operators and the core modeling principles of dynamic GNNs. They also come from two typical application areas for dynamic GNNs, traffic forecasting and video recognition, further supporting their relevance as representative benchmarks.

More details of the baseline models can be found in Section 2.4 and Section 4.2.

(2) Comparing our model with Dynamic GNNs to highlight the necessity of our design

In contrast, our model adopts a *hybrid design* rather than following a temporal- or structure-first design. This design reflects the targeted adaptations to the specific context of sales forecasting, and differs from typical Dynamic GNNs in three key areas:

- *Graph stability*: Graphs constructed from daily customer shopping journeys often exhibit high variability over time and contain noisy edges due to incidental co-clicks. To mitigate this issue, as discussed in Section 3.1, we first aggregate the graphs over a defined context window to obtain an average graph $\bar{A}$ (see Figure 3B in the manuscript). Then, as detailed in Section 4.1., we apply a filter that retains only edges appearing on at least th_{edge} days (about half the length of the context window) in this average graph. This strategy yields a stable representation of node neighborhoods. Thus, unlike dynamic GNNs that operate on a sequence of evolving graphs, our method uses a single, aggregated graph for learning.
- *Integration between GNN and RNN*: As shown in A2-1(E) (a rearranged version of Figure 2 from the manuscript for clearer comparison), our approach first performs temporal modeling using an RNN module, and then applies graph convolution on the aggregated graph using the final hidden states from the RNN. Thus, different from dynamic GNNs, which performs graph convolution at every time step, our model conducts graph convolution only once, at the end of the context window. Considering our graph contains millions of nodes, this design also improves computation efficiency in addition to model stability.
- *Unique GNN Modules*: As shown in A2-1(E), our model has two specialized GNN modules, one for predicting advance demand and the other for capturing bustiness, in addition to the RNN module that produces forecasts based on SKU time series. According to the forecasting literature, when a product has a stable sales history and no promotions are scheduled in the forecast horizon, the sales

history alone can provide sufficient information for an accurate forecast (Fildes et al., 2019). Thus, we treat the RNN forecasts as base forecasts, while the GNN outputs serve as corrections during atypical periods, such as promotions. This separation allows each module to focus on specific signals and helps prevent feature entanglement. For example, the burstiness prediction GNN can attend to sudden demand spikes from related SKUs, and it can be deactivated, i.e., producing near-zero forecasts, when no such signals are detected. This automatic activation mechanism is essential, as promotions are infrequent and do not uniformly impact all SKUs. Moreover, we keep the three prediction components separate to enhance the model’s interpretability.

To summarize, our model is specifically designed to tackle key challenges in the sales forecasting domain, including large-scale graphs with high level of variability and noise, complex product relationships and spillover effects, and the need for robustness and interpretability. In contrast, existing dynamic GNN models struggle to address these issues effectively, as reflected in their performance shown in Table 4 in the manuscript.

Appendix 3: Notations

		Notations	
M	Total number of nodes	T	A time period of length T that is used as a forecasting context window
$\mathbb{V}$	Node set	H	Prediction horizon length
q	Quantile	Q	Prediction quantile set
t	Time index	V_t	Node set at time t
i	Node index	v_i	Node (i.e., product-stage combination) with index i
E_t	Edge set at time t	G_t	Clickstream Graph at time t
$y_{i,t}$	Attribute of node v_i at time t	$w_{i,j,t}$	Weight of the edge from nodes v_i to v_j at time t
Y_i	Node time series that collects attributes of node v_i within time window T	$W_{i,j}$	Weight time series associated with the edge from v_i to v_j within time T
$\mathcal{Y}_{i,T+h}^{(q)}$	Prediction quantity at q quantile level at time $T+h$	Emv_i	Exponential moving average of Y_i with a smoothing factor λ
N_i^{Up}	Upstream neighbors for node i	N_i^{Down}	Downstream neighbors for node i
$b_{i,t}$	Burstiness for node i at time t	B_i	Burstiness scores series for nodes in time series Y_i
$NodeEmb_i$	Embedding of node i generated by processing Y_i through LSTM	$BaseForecast_i$	Base forecasts for node i based on node embedding $NodeEmb_i$
$BurstEmb_i$	Embedding of the burstiness of node i	$BurstForecast_i$	Forecasts for node i based on node burstiness $BurstEmb_i$
$NeighEmb_i$	Embedding of the neighbors of node i	$NeighForecast_i$	Forecasts for node i based on neighbor embedding $NeighEmb_i$
$W_{context}$	Weight matrix for step-specific context vector	β_i	Coefficient of gated mechanism to control inflation in forecasts based on neighbors
W_{key}	Key weight matrix in multi head attention	W_{output}	Weight matrix to output quantile forecast
W_{value}	Value weight matrix in multi head attention	W_{query}	Query weight matrix in multi head attention

Appendix 4: Comparing our Burst Prediction Module with Lappas et al., (2009)

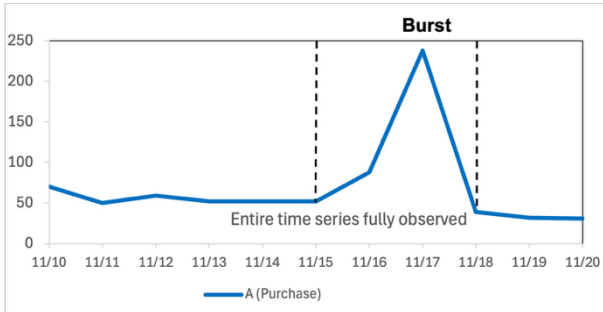
While our burst prediction module (Section 3.3) is motivated by Lappas et al. (2009), it fundamentally differs from it. Their method takes a *retrospective, non-parametric* approach to identify burst segments within a *single, fully observed* time series. In contrast, our method is *prospective and parametric*, as we train a GNN module with *learnable parameters* to predict future bursts based on *multiple related time series*. To clarify the difference, we briefly review this previous work.

Given a time series $[x_1, x_2, \dots, x_T]$, Lappas et al. (2009) measure burstiness of a segment $[r, l] \subseteq [1, T]$ as $\frac{\sum_{t=r}^l x_t}{\sum_{t=1}^T x_t} - \frac{l-r+1}{T}$, where $\frac{l-r+1}{T}$ expresses the expected behavior baseline, while $\frac{\sum_{t=r}^l x_t}{\sum_{t=1}^T x_t}$ represents the observed fraction. For example, in Figure 3(C), for the node time series $Y_3 = (9, 14, 17)$, the burstiness scores of each interval is $B_3 = \left(\frac{9}{9+14+17} - \frac{1}{3}, \frac{14}{9+14+17} - \frac{1}{3}, \frac{17}{9+14+17} - \frac{1}{3} \right) = (-0.108, 0.017, 0.092)$. The last two elements receive higher scores because their observed fractions exceed the expected baseline of $1/3$.

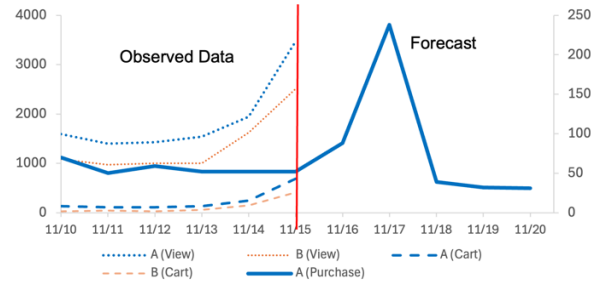
Building on the burstiness scores of a time series, Lappas et al. (2009) further propose an algorithm to identify a *maximal sum segment*, a contiguous subsequence whose burstiness score is not lower than any of its proper sub-segments or super-segments, as illustrated in Figure A4-1 (A).

Next, we discuss the differences between our approach and Lappas et al. (2009) :

- *Prospective vs. retrospective*: Lappas et al. (2009) take a retrospective approach to identify burst segments within a fully observed time series. In contrast, our method is prospective, as we aim to predict future bursts based on historical data. Figure A4-1 (B) illustrates the division of observed and unobserved data in our approach.
- *Parametric vs. non-parametric*: Lappas et al. (2009) use a non-parametric approach to search for maximum sum segments within a time series. In comparison, we adopt a parametric approach. We use the burstiness scores of a node and its neighbors as input features and train a model with learnable parameters to predict the node's future burstiness, as described in Equations (5)-(8) in Section 3 of the manuscript.
- *Single vs. multiple time series*: In general, bursts often resemble outliers within a time series. It can be difficult to predict outliers solely based on the time series itself. Hence, our approach relies on neighbors, i.e., multiple related time series, to provide richer contextual signals. In contrast Lappas et al. (2009) analyze individual time series.



(A) Lappas et al. (2009): Burst Segment Identification (Retrospective)



(B) Our Burst prediction (Prospective)

Figure A4-1: Comparing our burst prediction module with Lappas et al. (2009)

To summarize, while our work leverages an existing method to extract burstiness features, the problem we address and the methodology we employ are fundamentally different. To the best of our knowledge, this is the first approach to predict sales burstiness by incorporating information from related products.

Appendix 5: Elaboration of Gate Mechanism

Our model produces sales forecasts summing three effects for each SKU: its own time series trend (*BaseForecast*), advance demand from its related SKUs (*NeighForecast*), and its neighbor burst trend (*BurstForecast*). Since *NeighForecast* may be correlated with the other effects, we use a custom-designed gate mechanism to regulate *NeighForecast* in order to avoid inflating sales forecasts. Here we illustrate the computation of this gate mechanism during *training* and *test* stages in detail, using node v_3 as an example.

(1) Training stage

Assume we have the following node embedding, neighbor burstiness embedding, and neighbor embedding for v_3 (Purchase of SKU A), each with a dimension of d :

$$\begin{aligned} \text{NodeEmb}_3 &= (0.125, 0.002, \dots) && \text{(by Equation 3)} \\ \text{BurstEmb}_3 &= (0.204, 0.005, \dots) && \text{(by Equation 7)} \\ \text{NeighEmb}_3 &= (0.003, 0.025, \dots) && \text{(by Equation 14)} \end{aligned}$$

Each embedding enters a multilayer perceptron (MLP) network (Figure 2(B)) to generate forecasts independently, i.e.,

- *Base forecast* using node v_3 's time series embedding NodeEmb_i :

$$\text{BaseForecast}_3 = \text{MLP}(\text{NodeEmb}_3) \quad \text{(Equation 4)}$$
- *Burst forecast* using node v_3 's neighbor burstiness embedding BurstEmb_3 :

$$\text{BurstForecast}_3 = \text{Emv}_3 * \tanh(\text{MLP}(\text{BurstEmb}_3)) \quad \text{(Equation 8)}$$

where Emv_3 is the exponential moving average of node v_3 's time series, and $\tanh$ is the hyperbolic tangent function to produce a value between $[-1, 1]$.
- *Prediction of advance demand* based on node v_3 's neighbor embeddings NeighEmb_3 :

$$\text{NeighForecast}_3 = \text{MLP}(\text{NeighEmb}_3) \quad \text{(Equation 15)}$$

However, the base forecast and burst forecast may already account for advance demand signals, especially during promotion periods. For example, for the illustrative example shown in Figure 3, we plot the time series of v_3 's along with its most influential neighbor, cart of SKU, after scaling, shown in Figure A5-1. Both time series exhibit upward trends. As a result, the base forecast and neighbor forecast may each reflect this trend, and their combined effect could lead to an inflated sales prediction.

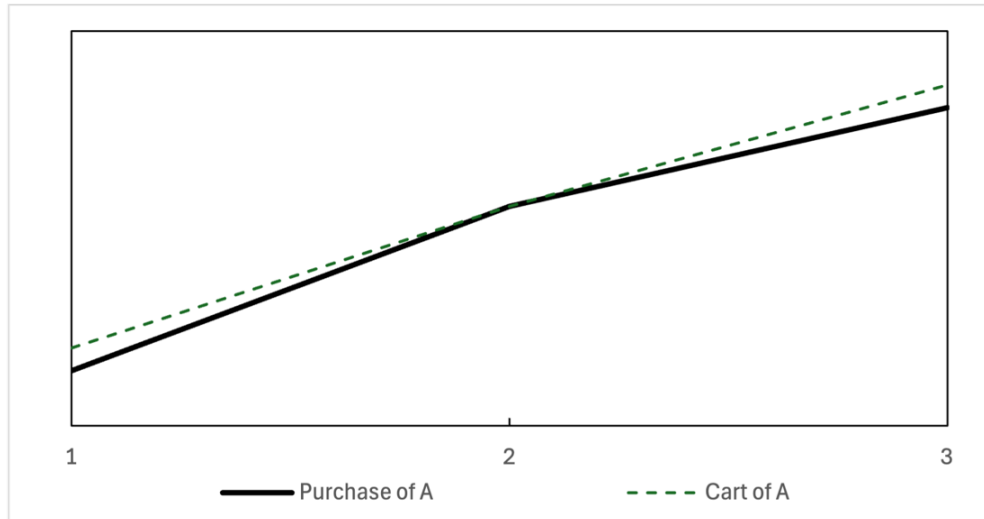


Figure A5-1: Time series of SKU A and its related products

Therefore, we dynamically compute coefficient β_i to regulate the influence of information from the neighbor context. The intuition is that by assessing the three embeddings, $NodeEmb_3$, $BurstEmb_3$, and $NeighEmb_3$, the model learns how much of the forecasts from the neighbor embedding ($NeighEmb_3$) can be incorporated into the final forecasts (Equation 17).

Assuming each embedding is a vector of d dimension, we concatenate them into a vector of $3d$, and multiply the concatenated vector by a tunable parameter $W_{NeighW} \in \mathbb{R}^{H \times 3d}$ to obtain a vector of $z_i \in \mathbb{R}^{H \times 1}$, where H is the length of forecast horizon. Next z_i is converted into a vector of probabilities by the Sigmoid function (Equation 16). These probabilities specify the percentages of $NeighForecast_3$ to be considered at each time step within the forecast horizon (Equation 17).

$$\beta_i = \frac{1}{1+e^{-z_i}}, \text{ where } z_i = W_{NeighW} \text{Concat}(NodeEmb_i, BurstEmb_i, NeighEmb_i) \quad (\text{Equation 16})$$

$$RegulatedNeighForecast_i = \beta_i * NeighForecast_i \quad (\text{Equation 17})$$

W_{NeighW} is tuned as other model parameters to minimize the quantile loss.

(2) Test stage

Once the model is trained, when testing with new data, β_i is directly calculated using the tuned parameter values W_{NeighW} and three embeddings calculated from their respective trained modules (see Equation 16).

Appendix 6: Metadata-enriched Model

Incorporating product metadata such as brand and category information could provide valuable predictive signals for sales forecasting. Our dataset contains product category (e.g., electronics, appliance) and brand (e.g., Apple, Samsung). However, data quality challenges are substantial. Table A6-1 shows more than one-third of products (386,299 in total) are missing either product category (38.08%) or brand (40.66%).

Metadata	Count	Product Missing Metadata	Missing Rate
Category	142	147,091	38.08%
Brand	5,384	157,080	40.66%

Table A6-1: Summary of Product Meta Features

Despite these limitations, we implemented the metadata-enhanced model that incorporates learned embeddings for brand and category information. Each product category or brand is represented as a dense vector of dimension 8, while a default embedding is assigned to products with missing metadata. The embedding dimension was selected based on empirical performance as well as feature cardinality: 142 unique categories and 5,384 unique brands (Table A6-1). Following the common heuristic that embedding dimensionality should lie between the fourth root and the square root of the vocabulary size, this choice provides a balance between representational capacity and computational efficiency.

The embeddings are processed through a dedicated MLP branch to learn metadata-specific representations for sales forecasting. At the final fusion stage, these metadata embeddings are combined with the outputs of the Burst detection module (which captures temporal burstiness patterns) and the Graph Neighbor module (which aggregates product interactions) to generate final forecasts.

We evaluated this enriched model using the same settings described in Section 4. Table A6-2 compares its performance with the original model. Incorporating product metadata reduces MSE by 2.5%, indicating that these features provide useful signals for absolute error reduction. However, SMAPE increases by 2.7%, suggesting that substantial missing values (>38%) may introduce noise that affects percentage-based error metrics, particularly for lower-quantity predictions.

Metric	Original Model	Metadata enriched Model	Improvement
MSE	428.09	417.43	+2.5%
SMAPE	32.08%	32.95%	-2.7%

Table A6-2: Performance Evaluation of Metadata-enrich Model

Appendix 7: Model Implementation and Experiment Setup

For fair comparison, we use the same parameter values for all components shared across the models. Table A7-1 summarizes the configuration details. As described in Section 4.1, we adopted time-based cross validation (Sun et al., 2022) to train and test the models. The models were trained using one month of time series data and tested on the subsequent month. During training, time series from 10% randomly selected SKUs were reserved for model validation. All models underwent at maximum 200 training epochs with a learning rate 0.001 following an early stopping procedure.

To address the data sparsity issue, we evaluated model performance only on SKUs with at least 14 days of purchase events during the training period. Meanwhile, we removed incidental edges that appear in fewer than th_{edge} days, where th_{edge} is set to approximately half the context window length.

Due to significant sales fluctuations during promotion periods, in order to respond to demand changes timely, we set the context window to around one week (i.e., [5,10] days) and forecast 1-5 days ahead. All models, except DeepAR, were trained with a quantile loss function targeting the 10th, 50th, and 90th percentiles. However, we added more quantiles for the experiments for evaluating quantile forecasts, as described in Section 5.6. DeepAR used a student T distribution due to its robustness to outliers. The Prophet model was trained independently for each SKU.

For the deep learning models, the dimension of LSTM layers was set to 50, which is sufficient for short time series with 1-3 input features (i.e., view, cart, and purchase counts). We tested other dimensions ranging from 20 to 100 and selected 50 based on the best validation performance. Similarly, all hidden layers were also set to a dimension of 50.

For the GNN models, we found one-layer graph convolution produced the best performance, and higher-order convolution did not yield significant improvement (see Table 7). In the self-attention module, we only enabled one-head self-attention, which proved sufficient given the short context window.

We used the neural forecast package of Nixtla (<https://github.com/Nixtla/neuralforecast>) to implement most of the benchmarking models. Our custom code can be shared upon request.

Parameters	Values	Models/Modules
Train/Test Split	Time-based nested cross validation (See Figure 5 and Section 4.1)	All
SKU Selection	Minimum 14 days of purchase events in the training dataset. 3445 SKUs in Oct. and 3251 SKUs in Nov. (See Section 4.1.)	All
Co-click threshold (th_{edge})	3; sensitivity study on $th_{edge} = 4$ (See Section 4.1.)	All
Context window (T) Forecast Horizon (H)	T = 7, H = 3; sensitivity study on T=(5,10) and H=(1,5)	All
Quantiles Q	Q = (0.1, 0.5, 0.9)	All except DeepAR and Prophet
LSTM hidden states	50	All LSTM layers
MLP hidden layer dimension	50	All MLP modules
Neighbors	First-tier neighbors; sensitivity study on two-tier neighbors	All GNN modules
Self-attention head	1; since the context window is short, 1 head is sufficient	All self-attention modules
Training epochs and learning rate	200 with early stopping Learning rate 0.001, Reduce On Plateau	All DL models
DeepAR target distribution	Student T	DeepAR
Prophet setup	Trained per SKU with all the data up to the forecast window	Prophet

Table A7-1: Model Configuration

Appendix 8: Post-hoc Analysis of Two Graph Components

As described in Equation (18), the forecast for a product i consists of three additive components:

- *BaseForecast_i* component is generated solely from the product’s own historical sales (i.e., a univariate forecast).
- *RegulatedNeighForecast_i* component incorporates information for related products, including view, cart, and purchase event counts, and is regulated by a learned attention coefficient $\beta_i \in (0,1)$, as shown in Equation (17). A larger β_i indicates that the model places greater attention on cross-product contextual information.
- *BurstForecast_i* component captures abrupt demand changes through a learned burst attention ratio $\lambda_i \in (-1,1)$, derived from aggregated neighbor burstiness signals, as shown in Equation (8). Positive λ_i increases the forecast to accommodate sudden demand surges, whereas negative λ_i reduces the forecast to capture post-promotion declines in demand.

Our core premise is that promotions alter consumer behavior, induce sudden demand spikes, and create cross-product spillover effects that cannot be fully captured by historical sales alone. As β_i and λ_i function as attention weights in our model, we therefore expect the model to assign substantially higher attention to neighbor context and burst adjustments during promotional periods, when such cross-product interactions and abrupt demand changes become more frequent.

To validate this mechanism, we extract the learned β_i and λ_i values used for forecasting product demands November and December and compute their averages across all products. Figure A8-1 presents the temporal patterns of these two attention weights.

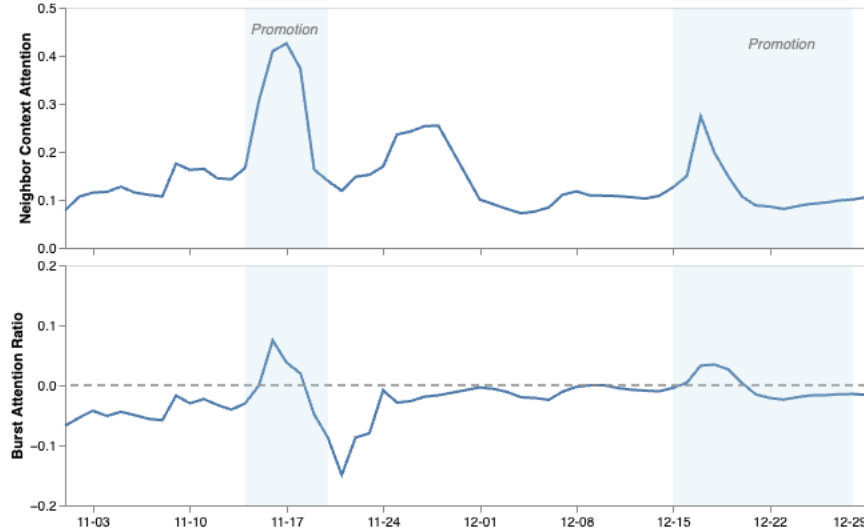


Figure A8-1: Average Neighbor Context Attention (β_i) and Burst Attention Ratio (λ_i)

As expected, the learned attention weights exhibit substantial and systematic shifts during promotion periods (November 14-20 and December 15-28, shaded in light blue). In particular, β_i increases significantly during promotions, indicating that the model relies more heavily on neighbor context when generating forecasts. For example, during the major November promotion period, the average β_i reaches about 0.4, compared to roughly 0.1 before the event. This pattern suggests that when demand is relatively stable, the univariate forecasting component is often sufficient, whereas promotions trigger stronger cross-product interactions that require contextual information from related products. Importantly, these

temporal patterns are not manually specified by the model design, but are learned from the training data. This suggests that the model automatically discovers the increased importance of neighbor context during promotional periods.

Similarly, the burst attention ratio λ_i is predominantly positive during promotion periods, reflecting strong upward demand trends. During the November promotion, *BurstForecast* increases forecasts by as much as 10% of the product’s exponential moving average. Because the EMA during promotional periods can be several times higher than regular sales, λ_i can produce a substantial forecast adjustment. In contrast, immediately after the promotion, λ_i often becomes negative, allowing the model to quickly adjust forecasts downward to account for reduced post-promotion demand and consumer stockpiling behavior (Hewage et al., 2022). For example, following the November promotion, *BurstForecast* reduces forecasts by up to 15% of the exponential moving average. Outside promotion periods, λ_i remains close to zero, indicating limited need for burst adjustment under stable demand conditions.

Overall, these findings provide interpretable post-hoc evidence that the proposed attention mechanisms operate consistently as designed. Specifically, the model adaptively increases attention to neighbor context and burst adjustments precisely during periods characterized by intensified product interactions and volatile demand dynamics, which helps explain the observed forecasting improvements.